Worley Data Engineering

None

None

None

Table of contents

1. Get Started 3
   1.1 Worley Data Pipelines 3
2. Process 5
   2.1 Data Engineering Concepts 5
   2.2 Data Engineering Process 8
3. Metadata 10
   3.1 Generic Curation Layer Job 10
4. Developer Tools 18
   4.1 Glue Job Local Development 18
   4.2 MWAA Local Development 19
5. Roadmap 20
6. Sourcing Documentation 21
   6.1 Overview of Assurance 21
   6.2 Overview of oraclegbs 24
   6.3 Overview of E3D 28
   6.4 Overview of sharepoint 33
   6.5 Overview of Ecosys 37
   6.6 Overview of Aconex 42
   6.7 Overview of Oracle P6 47
7. Data governance documnetation 51
   7.1 High level design 51
   7.2 Redshift Spectrum Table (External Table) 55
   7.3 RLS and DDL Support for External Tables in Amazon Redshift 56
   7.4 Redshift DB Admin playbook 57
   7.5 Accessing the platform 57
   7.6 Switching Identity types in Redshift Query Editor 58
   7.7 Redshift Internal Table Permission Grants 59
   7.8 Redshift Database Admin Set Up for DBT Data Modellers 63
   7.9 Adding Lakeformation admins 67
   7.10 Granting database access 67
   7.11 Grant Table access 0
   7.12 Column based Access 0
   7.13 RLS using data filters 0
   7.14 Redshift + QuickSight + Identity Center Integration Set Up 0

1. Get Started

1.1 Worley Data Pipelines

A repository that holds Worley's data pipelines and Worley's python helper package

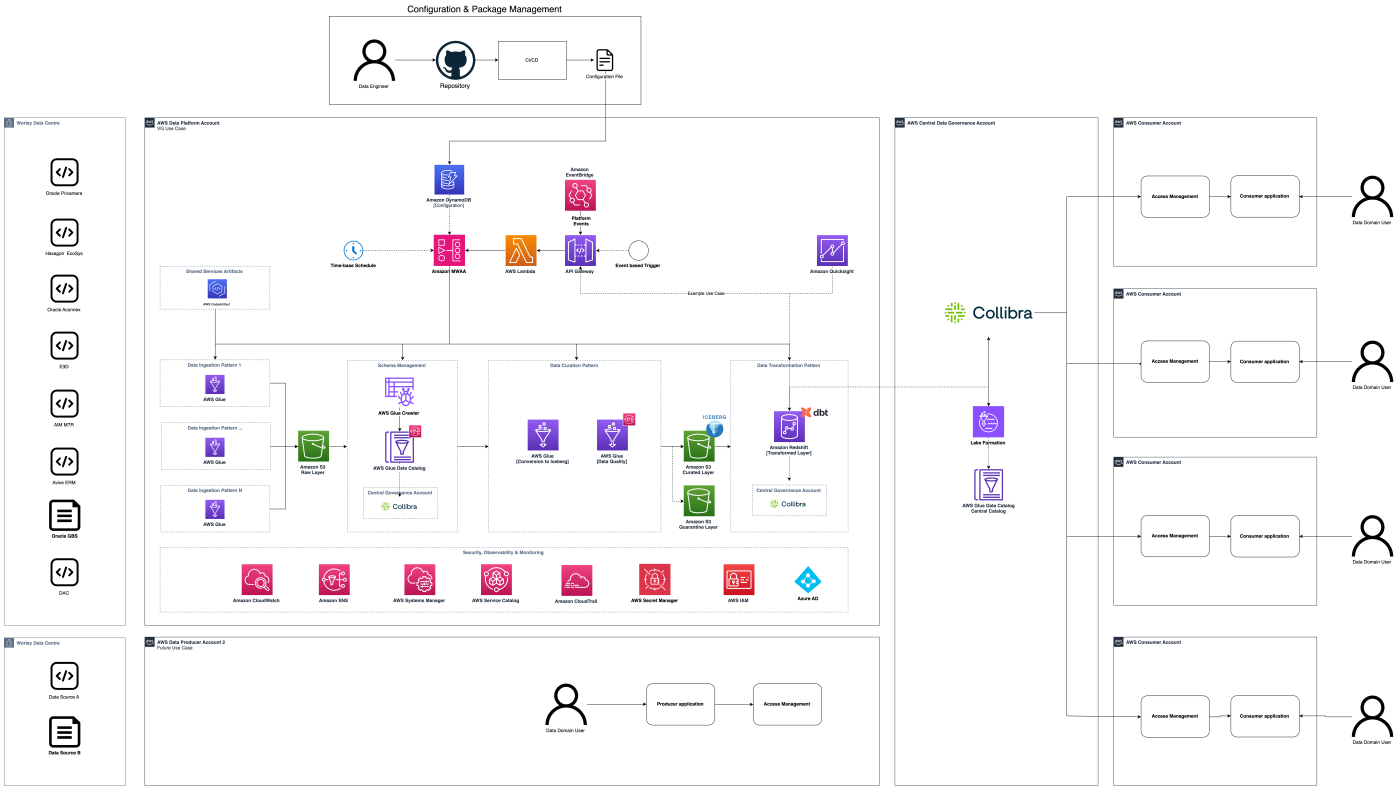
1.1.1 Tenets

Simple: A simple to use framework that puts the end user (data engineers & business owners) front and centre. The framework aims to remove the need for heavy DevOps/infrastructure and data engineering skills.

Open: The framework supports open-source code, open standards, and open frameworks. It also works with popular integrated development environments (IDEs), libraries, and programming languages.

Extensible: The framework is designed to be constantly developed and iterated. It takes a modular approach to enable data engineers to reuse components on new use cases

1.1.2 Architecture



Reference Architecture

1.1.3 Folder Structure

```
root
|   README.md
|   Makefile
|   requirements-docs.txt
|   mkdocs.yml
|
└──data_pipelines            - Worley's Data Pipeline definitions
|   |
|   |──0_sourcing            - Holds all data pipelines that moves data from `source` to the `raw` layer. Each subfolder refers to one application
|   |   └──aplication_1
```

```
|   |   └──aplication_2
|   |
|   └──1_curation              - Holds all data pipelines that moves data from the `raw` layer to the `curated` layer. Each subfolder refers to one table
|       └──table_1
|       └──table_2
|
└──worley_helper               - Worley's Python helper package
|
└──docs                        - Documentation folder based on `mkdocs`
```

2. Process

2.1 Data Engineering Concepts

2.1.1 Introduction

There are logical units of work that need to be developed when creating data products.

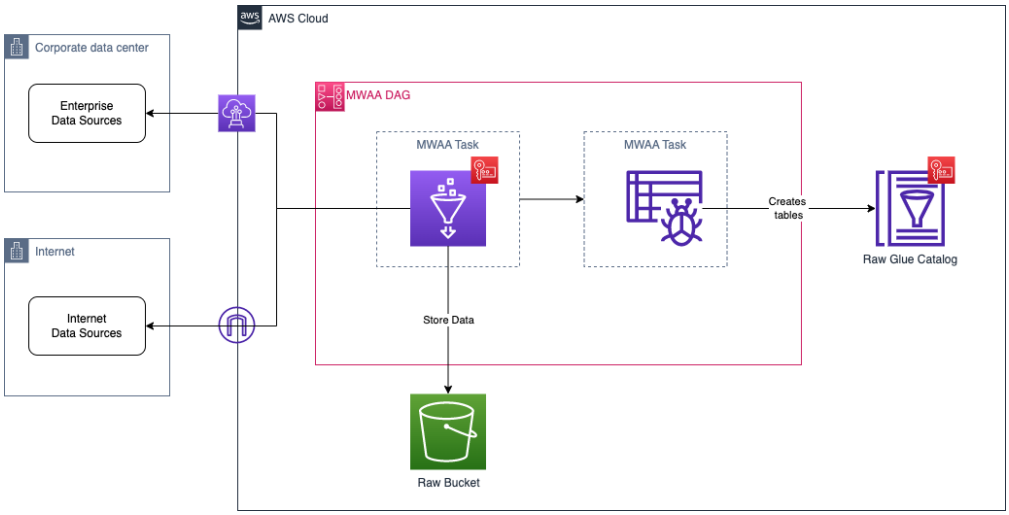
1. The platform needs to **ingest** data from source systems.
2. The platform needs to **curate** the ingested data to make it readily available for downstream consumers.
3. The platform needs to **model** the curated data to embed business rules and definitions
4. The platform needs to **serve** the available data to all platform users.

All of the above need to happen in a **secure** and **governed** way.

```
graph LR
  A[Source] --> B[Ingest/Source];
  B --> C[Curate];
  C --> D[Transform/Model];
  D --> D;
  D --> E[Serve];
```

2.1.2 Source/Ingest

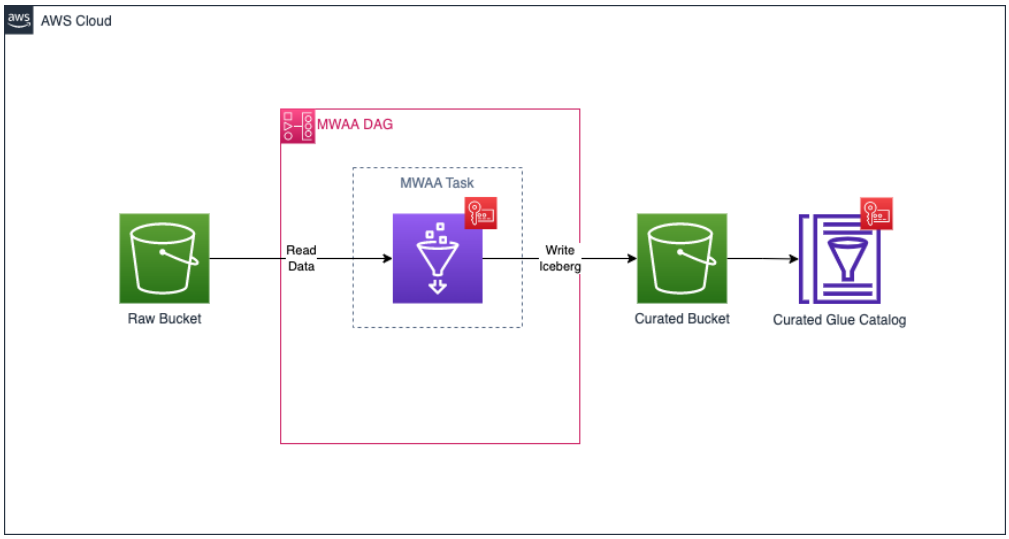
This step refers to bringing data from the Source (Databases, APIs, CSV, etc) to the Raw layer.



Data Ingestion/Sourcing

2.1.3 Curation

This steps refers to curating the data available in the `raw` layer.



Data Curation

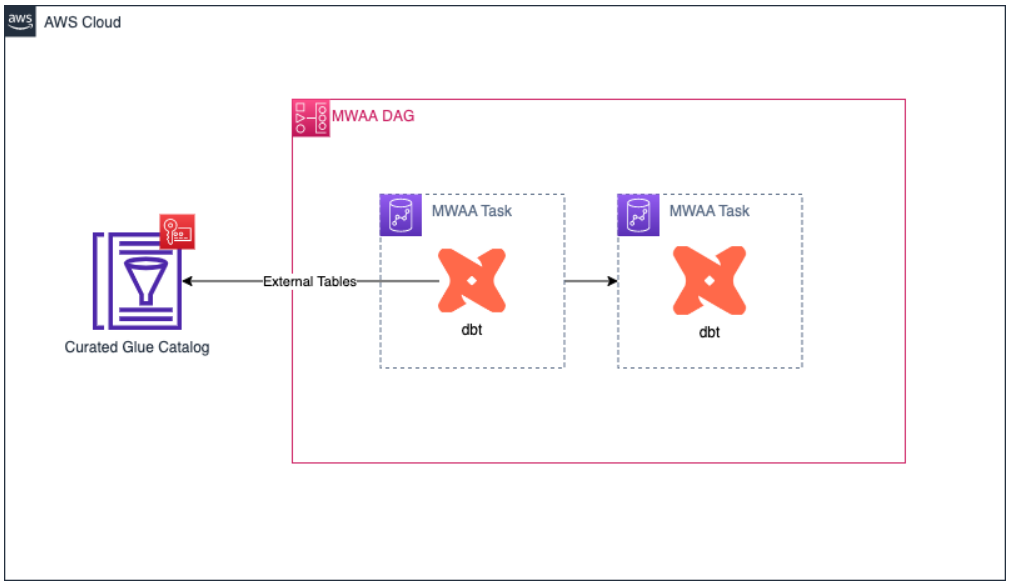
The data curation process can involve multiple steps such as:

1. Conversion to Iceberg **(mandatory)**
2. Data integrity checks **(optional)**
3. Light -table level- transformations **(optional)**
4. SCD Type I/II **(optional)**

As time progresses the curation process will mature.

2.1.4 Transformation/Model

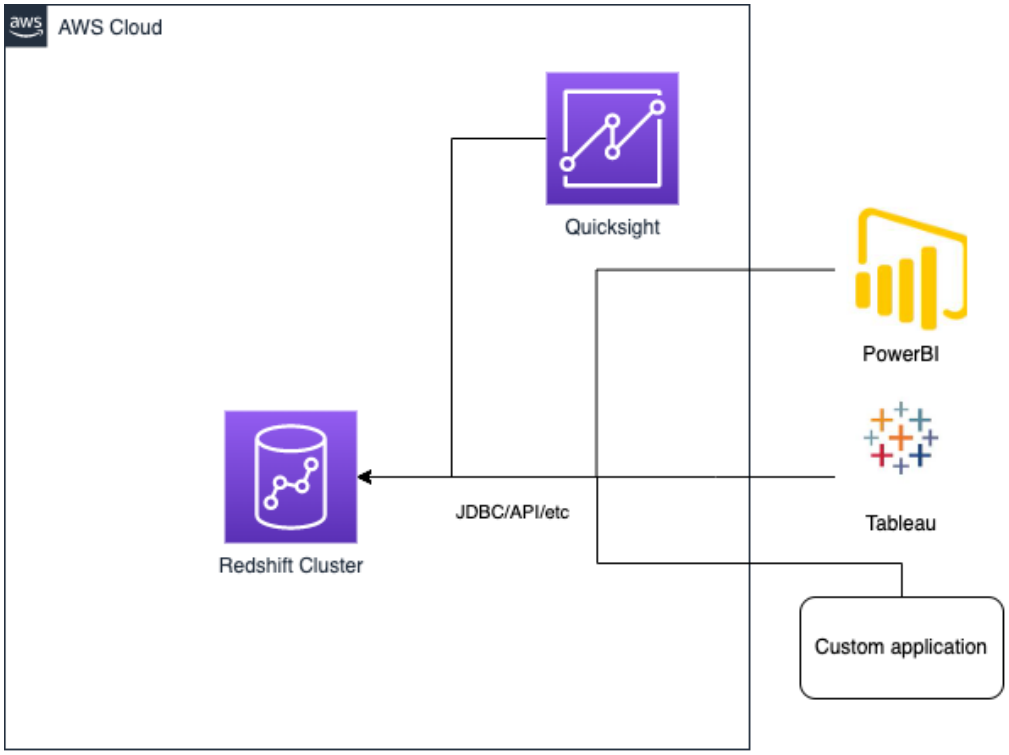
This steps refer to transforming the data to adhere to the business model.



Data Transformation

2.1.5 Serve

> **Warning**
>
> This is under construction



Data Serving

2.2 Data Engineering Process

There are three components to the platform

1. AWS Infrastructure
2. Data & Analytics Resources
3. Helper Utilities

To develop a new data product, you need to firstly deploy the required AWS resources (AWS Glue Database, AWS Glue Jobs, AWS Glue Crawlers, AWS Glue Connection*). The combination of these resources gives you the tools to be able to source a new dataset, curate it, and make it available to your Data Warehouse.

Then, you develop your data engineering workload (AWS Glue Jobs, `dbt` models or reporting Dashboards). Here you should aim to leverage the Helper utilities to speed up development.

Finally, you circle back any new addition to the helper utilities

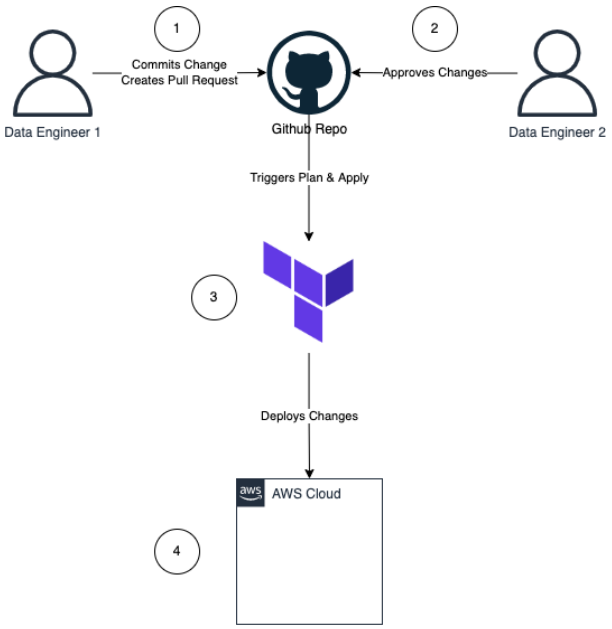
DATA PRODUCT LIFECYCLE

```
graph LR
  A[Define Business Need] --> B[Create/Deploy AWS Resources];
  B --> C[Create Data Engineering Resources];
  Z[Helper Library] -->|Use Helpers| C;
  C --> E[Publish Data Product]
```

2.2.1 DevOps Process - Automation to Deploy AWS Resources

DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity: evolving and improving products at a faster pace than organizations using traditional software development and infrastructure management processes. This speed enables organizations to better serve their customers and compete more effectively in the market.

Below is a high level diagram of the process:



DevOps Process

The flow is the following:

1. The Data Engineer makes the required changes to the infrastructure to create their desired AWS resources using Terraform. They then commit the changes to the repository and raises a Pull Request (PR).
2. A team member peer reviews the Pull Request and `approves` it. They can also `request for changes` or reject it. This gate is used to inject proper governance controls
3. Once the PR is approved and merged, an automation process is triggered to initiate the deployment of the resources by leveraging Terraform Cloud
4. Terraform Cloud runs the `plan` command and when successful, runs the `apply` command. After this, the Data Engineer has their set of Enterprise approved resources.

2.2.2 DataOps - Automation to Deploy Data Engineering Resources

Data Engineering resources refer to the different components that are required to perform `ETL`/`ELT` workloads and/or to develop reporting products.

The current data platform includes automation to allow developers to focus on fulfilling business needs and not worry about their process all whilst mantaining governance and controls.

3. Metadata

3.1 Generic Curation Layer Job

The implementation of a Generic Curation Layer job enables the transition from traditional Data Lake to an advanced Data Lakehouse architecture.

At it's core the generic curation job moves data from the Raw Data Lake layer to Iceberg tables, a key component of the Data Lakehouse.

The key ways in which the Generic Curation Layer job enables this transition include:

1. Data Standardization and Governance : By applying schema definitions , the job ensures data consistency
2. Simplified Transformation Execution : The Generic Curation Layer job leverages the metadata to execute the necessary data transformations as the data is moved from the Data Lake to the Iceberg tables.
3. Flexible Loading Modes : The Generic Curation Layer job provides the capability to load data into the Iceberg tables using different modes (e.g., full load, incremental load etc) based on the characteristics of the source data.
4. Time Travel and Snapshotting : Iceberg tables, the target of the curation layer job, provide time travel capabilities & snapshotting features which enables efficient version control and data rollbacks
5. Unified Data Access : By moving data into Iceberg tables, the curation layer job enables an unified data access layer within the Data Platform
6. Interoperability : The Iceberg table format, which the curation layer job utilizes, ensures interoperability across different data processing frameworks and tools.

3.1.1 Metadata Configuration

The Generic Curation Job leverages the helper library that provides a wrapper on top of common functionality to process data within the AWS Data Platform. It's main purpose is to read data from the `raw` layer and move it to the `curated` layer (i.e. Iceberg Table). This process is defined for each table to allow the Data Engineer to declaratively run the data pipelines.

The configuration file has the following goals:

1. Reuse logic across different data assets
2. Minimise the amount of code the data engineer needs to develop
3. Mantain governance by version controlling the file
4. Provide flexibility to cater for a wide spread of use cases.

The file is composed of the following main sections:

Section	Description
`source`	Defines the location of the source data assets.
`transforms`	Defines a list of transformation to be run against the source
`target`	Defines the end location of the data asset.
`table_schema`	Define the expected schema of the data asset

Source

This section has various properties for reading data from Raw Layer (i.e. S3 Buckets) :

- name : The name of the specific table within the broader data source landscape.
- compute_engine : The compute engine defines the underlying technology used for processing the data. Supported values are **spark**.
- spark_options : The format of the source data is crucial for determining the appropriate read and processing mechanisms.
- format : The format of the source data is crucial for determining the appropriate read mechanism. Supported values are **parquet**
- glue_options : The various glue configurations needed while reading the data from the raw layer.
- connection_type : The type of source within which files of the raw layer are present. Supported values are **s3**
- connection_options : The connection options provide the necessary details for accessing the source data, such as the S3 bucket and prefix.
- format : The format of the source data is crucial for determining the appropriate read mechanism. Supported values are **parquet**
- format_options: This property corresponds to format_options parameter when creating a Glue DynamicFrame from a data source and it allows you to specify additional options. The significance and usage of format_options depend on the specific data format you're working with.
- mergeSchema : Specifies whether to merge the schema of the data if the data files havedifferent but mutually compatible schemas. Supported values are **true** or **false**
- transformation_ctx : The transformation context helps to track the specific processing steps applied to the data files, which is useful for maintaing the job bookmarks for tracking processed files.

Transforms

This section has various properties that determines transformation to be run against the source:

- transform : rename_columns
- rename_column : Renaming columns is necessary to align the source data with the desired target schema and naming conventions. Supported values are **true** or **false**
- transform : select_columns_from_config_file
- select_columns : Selecting specific columns from the source data based on a configuration specified within **columns** section to ensure the target schema includes only the necessary data. Supported values are **true** or **false**
- transform : change_data_types
- change_types : Changing the data types of the source data for ensuring the data is stored and processed in the appropriate format as per the target table. Supported values are **true** or **false**
- transform : add_run_date
- column_name : Adding a run date column is useful for tracking the processing timeline and enabling time-based analysis or partitioning.
- date_format : The date format ensures the run date column is stored in the desired format.
- transform: custom_sql
- sql: The custom SQL transformation allows for more complex data manipulations and enrichments, which can be crucial for processing the data for the target table.
- temp_view_name : The temporary view name helps to manage the intermediate data processing steps.

  For example, the custom sql transformation can be leveraged to typecast a field with a specific date format (like "dd-MM-yy") with below SQL.

```
SELECT datasource_num_id, br_key, gbs_key, from_currency, to_currency, conversion_type, conversion_rate, 'oracle_gbs' as SOURCE_SYSTEM_NAME, to_date(month_start_date, 'dd-MM-yy') AS month_start_date, to_date(month_end_date, 'dd-MM-yy') AS month_end_date FROM temp_df_static
```

Note: If there are specfic transformations which are not needed for your table, you can skip corresponding section.

Table Schema

This section has various properties for defining the expected schema of the table or data asset:

- schema_properties : The various porperties of the the table schema can be specifed here.
- enforce : Enforcing the schema ensures the data written to the target table matches the expected structure. Supported values are **true** or **false**
- primary_key: Defining the primary key column(s) based on which data would be handled in the target table. If there are multiple columns, specify a list of columns as per target table schema.
- rename_columns: Significance: Renaming columns during the schema definition aligns the target table structure with the desired naming conventions. Supported values are **true** or **false**
- columns : The columns of the table are specified within the columns section.
- column_name : The column name defines the specific attribute nameof the data being stored in the target table.
- column_data_type : The data type of the column in the target table.
- nullable : The nullable property specifies whether the column can contain null values. Supported values are **true** or **false**
- comment : The column comment provides additional context and documentation about the attribute.
- data_classification: The data classification helps to identify the sensitivity or confidentiality of the data.
- raw_column_name : The raw column name within the raw data location.

Note: If no column renames and data type changes are needed from raw layer schema, you skip the columns section.
Note: In case, a specific column from the raw layer isn't present in the data but is present within column configuration, a corresponding curated column would be created for raw column and NULL values will be added

Target

This section has various properties for the target table or the data asset:

- name : The name of the data asset that helps to identify the table within the broader source ecosystem.
- compute_engine: The compute engine defines the underlying technology used for writing the data to the target. Supported values are **spark**
- drop_duplicates : This property defines whether duplicate rows have to be dropped while writing to the target table. Even if one of the attrbiutes, have a different values within the record, it wouldn't be dropped. Supported values are **true** or **false**
- iceberg_properties : The various properties of the target iceberg table.
- database_name : The database name specifies the location where the Iceberg table will be created, which is important for organizing and managing the data within the broader data infrastructure.
- table_name: The table name defines the specific Iceberg table that will be the target for the data.
- iceberg_configuration : The various iceberg configurations.
- create_table : This property defines whether the Iceberg table can be created if it doesn't exist. Supported values are **true** or **false**
- iceberg_catalog_warehouse : The Iceberg catalog warehouse location specifies where the Iceberg table metadata & data is stored, which is important for enabling the table to be accessed and queried.
- table_properties : The various configurations needed while creating the Iceberg Table.
- write.format.default: The data format of the files in the storage location. Supported values are **parquet**
- format-version : The format version specifies the version of the Iceberg table format. The latest version is **2**
- cdc_properties : The various configurations which need to be specified when the load type is **cdc**. For other load types, this section can be skipped.
- cdc_operation_column_name : The column name within the incoming data from raw layer which consist of the operation performed.
- cdc_operation_value_map : The value indicatorvfor different types of operations performed.
- insert : The value of the cdc_operation_column_name for the new inserted rows.
- update : The value of the cdc_operation_column_name for the updated rows.
- delete : The value of the cdc_operation_column_name for deleted rows.

  Please note that the value specifed for each operation is case sensitive.
- primary_constraint_properties : The various configurations which need to be specified to enforce deduplication on the primary key attribute.
- enforce_primary_constraint : This property defines whether the primary attribute deduplication needs to be enforced or not. Supported values are **true** or **false**
- timestamp_column_name : The name of the timestamp column based on which the latest record would be selected.
- load_type : The load type specifies the data loading strategy into the target Iceberg Table. The following load_types are supported:
- full_load : Truncate the data in target table and re-load with incoming data from raw layer
- incremental :
- This load type is to be used when the incoming data from raw layer is a point in time full refresh i.e. it consists of unchanged records, records which added or updated. Data is loaded to target table in **SCD Type 2** manner.
- Records which exist in the target table and aren't present in incoming data are considered as deleted records and will be soft deleted in the target table.
- This load type leverages the primary key to identify new version and old version of the record. If a primary key is not present, a row hash is used as the primary key.
- The SCD Type 2 behaviour may not be consistent if row hash is used as primary key.
- cdc :
- This load type is to be used when the incoming data from raw layer consists of **changes** which happened from the previous run.
- A status column defined (within the cdc_properties) indicates the type of operation i.e. insert, update or delete.
- Data is loaded to target table in **SCD Type 2** manner.
- The CDC load type uses the **cdc_properties** section and **primary_key** section. Data load would fail if either of these are not specified.
- incremental_no_delete :
- This load type is to be used when the incoming data from raw layer consists of changed or updated. Data is loaded to target table in **SCD Type 2** manner.
- This load type leverages the primary key to identify new version and old version of the record.
- This load type would not need a status column to derive the nature of change
- This load type would not account for records deleted in the source system.

- append : Append the incoming data from raw layer to target table.

3.1.2 Roadmap

- Data Quality Rules Integration : Enabling users to define and apply data quality rules directly within the curation layer job, strengthening the overall data quality and reliability.
- Project-level Data Loading : Providing the ability to load data into Iceberg tables at the project level, offering more granular control and flexibility.
- Automated Schema Evolution : Streamlining the process of schema changes, such as adding, dropping, or renaming columns, to reduce the manual effort required and ensure seamless data evolution.
- Iceberg Table Maintenance Operations : Integrating capabilities to perform maintenance tasks on Iceberg tables, such as expiration, and optimization, to ensure the efficient and sustainable management of the Data Lakehouse.

3.1.3 Example Configuration File

```
```
Sample YAML to showcase reading from parquet files and writing to Iceberg
Define Data Source
source:
  name: activitycode
  compute_engine: spark
  spark_options:
    format: parquet
  glue_options:
    connection_type: "s3"
    connection_options: "s3://<data_location>/{source_system_id}/{input_file}/"
    format: "parquet"
    format_options:
      mergeSchema: "true/false"
    transformation_ctx: "{source_system_id}-{input_file}"

Define standard transformations
transforms:
  - transform: custom_sql
    temp_view_name: my_table
    sql: "select * from my_table"
  - transform: add_run_date
    column_name: execution_date
    date_format: yyyy-MM-dd
Define Target
target:
  name: <sample_name>
  compute_engine: spark
  iceberg_properties:
    database_name: <curated_glue_database>
    table_name: <curated_glue_database_table>
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: <curated_data_location>/{source_system_id}/{input_file.lower()}/
      table_properties:
        write.format.default: parquet
        format-version: "2"
  spark_options:
    format: iceberg
    options:
      path: "s3://<curated_data_location>/{source_system_id}/{input_file}"
```
```

3.1.4 Example Target Section for different Load scenarios

- Delete the existing data in target table and load the incoming records

```
  target:
compute_engine: spark
iceberg_properties:
  database_name: worley_datalake_sydney_dev_glue_catalog_database_spel
  iceberg_configuration:
    create_table: true
    iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/spel/T_Cable/
    table_properties:
      format-version: '2'
      write.format.default: parquet
  table_name: curated_t_cable
load_type: full_load
name: curated_t_cable
spark_options:
  format: iceberg
```

```
  options:
    path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/spel/curated_t_cable
```

• Append records in incoming dataset to the target table

```
  target:
compute_engine: spark
iceberg_properties:
  database_name: worley_datalake_sydney_dev_glue_catalog_database_spel
  iceberg_configuration:
    create_table: true
    iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/spel/T_Cable/
    table_properties:
      format-version: '2'
      write.format.default: parquet
  table_name: curated_t_cable
load_type: append
name: curated_t_cable
spark_options:
  format: iceberg
  options:
    path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/spel/curated_t_cable
```

• **CDC based load** into target table (SCD Type 2) driven by a CDC Staus column indicating the type of operation

```
target:
cdc_properties:
  cdc_operation_column_name: status
  cdc_operation_value_map:
    delete: D
    insert: M
    update: M
compute_engine: spark
iceberg_properties:
  database_name: worley_datalake_sydney_dev_glue_catalog_database_erm
  iceberg_configuration:
    create_table: true
    iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/erm/PublishPLItem/
    table_properties:
      format-version: '2'
      write.format.default: parquet
  table_name: curated_erm_publishplitem
load_type: cdc
name: curated_erm_publishplitem
primary_key: primarykey
spark_options:
  format: iceberg
  options:
    path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/erm/PublishPLItem
```

• Upsert (Update & Insert) load of the incoming records into target table (SCD Type 2) where **source pushes only updated and new records**. The below example shows the usage of drop_duplicates parameter for dropping exact duplicate records within the incoming records

```
target:
  name: curated_aconex_workflow
  compute_engine: spark
  drop_duplicates: true
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_aconex
    table_name: curated_workflow
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/aconex/workflow/
      table_properties:
        write.format.default: parquet
        format-version: '2'
  load_type: incremental_no_delete
  primary_key: [WorkflowId,projectid,Assignee_UserId]
  spark_options:
    format: iceberg
    options:
      path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/aconex/workflow
```

• Performing a full refresh load into target table (SCD TYpe 2) with the incoming records where **source pushes the complete data**. The below example shows the usage of enforcing primary constraint property to ensure only the latest record (based on timestamp column) is considered if there are duplicates on primary key attributes

```
target:
  compute_engine: spark
  load_type: incremental_no_delete
  name: curated_ecosys_project_list_bkp_merge
  primary_constraint_properties:
    enforce_primary_constraint: true
    timestamp_column_name: SystemDate
  iceberg_properties:
```

```
    database_name: worley_datalake_sydney_dev_glue_catalog_database_ecosys
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: >-
        worley-datalake-sydney-dev-bucket-curated-xd5ydg/ecosys/project_list_bkp_merge/
      table_properties:
        format-version: '2'
        write.format.default: parquet
    table_name: curated_project_list_bkp_merge
  spark_options:
    format: iceberg
    options:
      path: >-
        s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/ecosys/project_list_bkp_merge
```

4. Developer Tools

4.1 Glue Job Local Development

You can develop AWS Glue Jobs locally by using AWS Glue Docker images hosted on Docker Hub or the Amazon Elastic Container Registry (Amazon ECR) Public Gallery. The Docker images help you set up your development environment with additional utilities. You can use your preferred IDE, notebook, or REPL using the AWS Glue ETL library.

4.1.1 Instructions

To run Glue Jobs locally, please follow these steps

1. Pull the image from the Registry

```
docker pull amazon/aws-glue-libs:glue_libs_4.0.0_image_01
```

2. Add the required environment variables. These will passed on to the Docker container

```
export AWS_ACCESS_KEY_ID="YOU_AWS_ACCESS_KEY_ID"
export AWS_SECRET_ACCESS_KEY="AWS_SECRET_ACCESS_KEY"
export AWS_SESSION_TOKEN="AWS_SESSION_TOKEN"
export JUPYTER_WORKSPACE_LOCATION="/YOU/FULL/PATH/TO/LOCAL/FILES"
```

3. Run the Docker image

```
docker run -it -v ~/.aws:/home/glue_user/.aws -v $JUPYTER_WORKSPACE_LOCATION:/home/glue_user/workspace/jupyter_workspace/ -e
AWS_ACCESS_KEY_ID=$AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY=$AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN=$AWS_SESSION_TOKEN -e DISABLE_SSL=true -e
DATALAKE_FORMATS=iceberg -e --rm -p 4040:4040 -p 18080:18080 -p 8998:8998 -p 8888:8888 --name glue_jupyter_lab amazon/aws-glue-libs:glue_libs_4.0.0_image_01 /
home/glue_user/jupyter/jupyter_start.sh
```

4.1.2 Limitations & Documentation

- AWS Guide
- Local Development Limitations

4.2 MWAA Local Development

You can develop MWAA DAGs by using MWAA local runner. The following repository provides a command line interface (CLI) utility that replicates an Amazon Managed Workflows for Apache Airflow (MWAA) environment locally.

4.2.1 Instructions

To avoid repetition, please refer to the README file in the aws-mwaa-local-runner repository. In a nutshell, you will need `Docker` installed in your local environment.

4.2.2 Accessing AWS Resources

Please note that your local environment will require appropiate access to AWS resources. Please follow the instructions in the official Airflow Documentation page to use your AWS credentials with local MWAA

5. Roadmap

The Worley Data Engineering Roadmap will exist here

6. Sourcing Documentation

6.1 Overview of Assurance

Assurance is a datasource that is SFTP server based. The datasource consists of CSV files. In an ideal scenario, Assurance generates 148 CSV files. The datasource wraps all the CSV files into a single zip archive. Assurance has a global namespace and is not project specific unlike E3D or Sharepoint. The schema (headers of CSV files) of the 148 files can be different from each other.

Currently, the CSV files are uploaded manually to a S3 bucket and these files are consumed by raw layer and curation layer Glue jobs. However, future scope may involve logic (possibly using AppFlow connector and Airflow Dag combination) that fetches the zip archive from SFTP and places the unzipped CSV files on S3.

The CSV files are processed sequentially within the raw layer Glue job and diagnostic information such as missing rows or empty CSV files are logged using CloudWatch logs. The CSV files are processed by a generic raw layer Glue job that uses metadata configuration to get contextual information of the CSV files. The generic Glue job is designed to handle any other datasources that generate CSV files similar to Assurance. The curation layer Glue job is also generic in nature and handles the transformations using the metadata configuration of the curation layer.

6.1.1 Metadata files

The table below explains the purpose of some the important parameters.

Filename	Description
`sampling_fraction and sampling_seed`	These parameters are used to collect a sample set of data which is further used to detect schema changes.
`replace_non_alphanumeric_with_underscore`	If the raw layer columns contain non alphanumeric characters they will be replaced by an underscore `_`.
`is_fixed_file_name, file_prefix_length, file_suffix_length`	In some cases the file name may end with varying date and time stamps. In order to generically identify and parse files these parameters are used.
`separator`	The CSV files can have various separators such as comma-separated, tab-separated, or pipe-separated.

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```yaml
---
SourceSystemId: assurance
MetadataType: csv#assurance
aws_region: ap-southeast-2
Env: dev
is_active: 'y'
job_parameter:
  bucket_name: <raw-layer-bucket-name>
  csv_files_path: assurance/csv/
  drop_duplicate_columns: true
  drop_duplicate_rows: true
  parquet_files_path: assurance/parquet/
  raw_files_path: assurance/raw/
  replacement_char: _
  replace_non_alphanumeric_with_underscore: true
  replace_non_printable_ascii_with_underscore: false
  sample_data_location: assurance/data_sampling
  sampling_fraction: '0.5'
  sampling_seed: 42
  specialchars_to_be_replaced_in_columnnames: []
  csv_files:
    - filename: dataextract_rt_action_factors_training_and_competence_20240506_1944.csv
      file_prefix_length: 0
      file_suffix_length: 14
      is_fixed_file_name: false
      multiline: true
      quote_character: '"'
      separator: '|'
      tablename: action_factors_training_and_competence
      with_header: true
```

```
    - filename: dataextract_rt_assessment_finding_20240506_1944.csv
      file_prefix_length: 0
      file_suffix_length: 14
      is_fixed_file_name: false
      multiline: true
      quote_character: '"'
      separator: '|'
      tablename: assessment_finding
      with_header: true

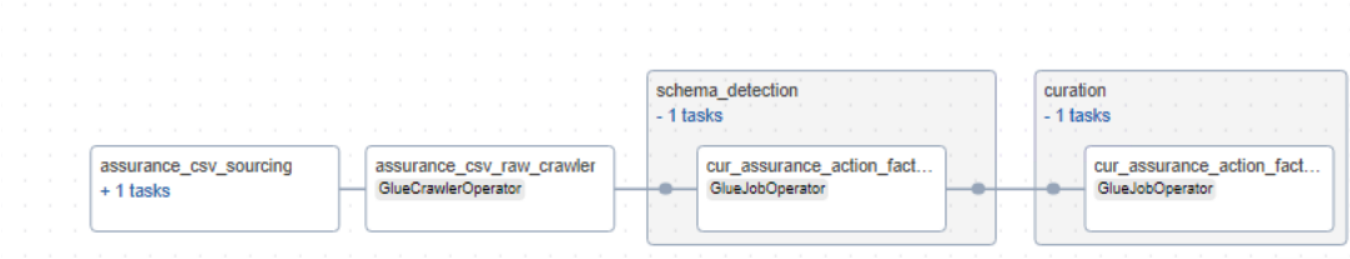
name: assurance
Namespace: global
```

Curation Layer Metadata Example

```
---
SourceSystemId: assurance_curated
MetadataType: curated#assurance#action_factors_training_and_competence#job#iceberg
source:
  compute_engine: spark
  glue_options:
    connection_options: s3://<raw-layer-bucket-name>/assurance/parquet/action_factors_training_and_competence.parquet/
    connection_type: s3
    format: parquet
    transformation_ctx: action_factors_training_and_competence
    format_options:
      mergeSchema: true
  name: action_factors_training_and_competence
  spark_options:
    format: parquet
table_schema:
  columns:
  - column_data_type: decimal(32,0)
    column_name: dwk_action
    comment: dwk_action
    data_classification:
    nullable: false
    raw_column_name: dwk_action
  - column_data_type: string
    column_name: factors_training_and_competence
    comment: factors_training_and_competence
    data_classification:
    nullable: true
    raw_column_name: factors_training_and_competence
  - column_data_type: string
    column_name: li_factors_training_and_competence
    comment: li_factors_training_and_competence
    data_classification:
    nullable: false
    raw_column_name: li_factors_training_and_competence
  - column_data_type: decimal(32,0)
    column_name: lo_factors_training_and_competence
    comment: lo_factors_training_and_competence
    data_classification:
    nullable: true
    raw_column_name: lo_factors_training_and_competence
  - column_data_type: timestamp
    column_name: bms_lastmodified
    comment: bms_lastmodified
    data_classification:
    nullable: true
    raw_column_name: bms_lastmodified
  schema_properties:
    enforce: true
    rename_columns: true
    primary_key: [dwk_action,li_factors_training_and_competence]
target:
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_assurance
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: <curated-layer-bucket-name>/assurance/action_factors_training_and_competence/
      table_properties:
        format-version: '2'
        write.format.default: parquet
    table_name: curated_action_factors_training_and_competence
  name: curated_action_factors_training_and_competence
  spark_options:
    format: iceberg
    options:
      path: s3://<curated-layer-bucket-name>/assurance/action_factors_training_and_competence
  load_type: incremental
  primary_key: [dwk_action,li_factors_training_and_competence]
transforms:
- rename_column: true
  transform: rename_columns
- select_columns: true
  transform: select_columns_from_config_file
- change_types: true
  transform: change_data_types
```

```
- column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
  transform: add_run_date
- sql: SELECT *, 'assurance' as SOURCE_SYSTEM_NAME FROM temp_df_static
  temp_view_name: temp_df_static
  transform: custom_sql
```

6.1.2 Assurance Pipeline



The assurance pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process CSV files.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.1.3 Key Files Handling Assurance Files Processing

Filename	Description
`assurance_csv_pipeline.py`	The Airflow DAG pipeline for handling Assurance files processing.
`convert_csv_and_xlsx_to_parquet.py`	Generic Glue job that converts CSV and XLSX files to Parquet format. Expects the following gluejob input parameters: `--source_name:assurance, --metadata_type:csv#assurance, --function_name:csv, --metadata_table_name:<dynamodb_tablename>`
`schema_change_detection_generic.py`	Generic Glue job that detects schema changes between current and previous versions. Expects the following gluejob input parameters: `--catalog_db:<glue_catalogue_db_for_assurance>, --table_name:<raw_table_name_of_the_csv_file>`
`raw_curated_generic.py`	Generic Glue job that creates iceberg tables. Expects the following gluejob input parameters: `--source_system_id: assurance_curated, --metadata_type: curated#assurance#<table_name_of_the_csv_file>#job#iceberg, --metadata_table_name:<dynamodb_tablename>`.

6.1.4 Process to onboard new files to Assurance

- Add the metadata of the new file to raw layer metadata file.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the curation layer table to the Airflow DAG `assurance_csv_pipeline.py`.

6.1.5 Future Scope

- Connector functionality to fetch ZIP files from SFTP server, unzip it and upload it to S3.
- Customizing raw layer metadata configuration to dynamically decide if Glue job processing should exit in the event of errors originating from one or more CSV files or gracefully catch the errors and continue processing rest of the files.

6.2 Overview of oraclegbs

Oraclegbs is a datasource that is SFTP server based. The datasource consists of CSV files. The schema (headers of CSV files) of the files can be different from each other.

Currently, the CSV files are uploaded manually to a S3 bucket and these files are consumed by raw layer and curation layer Glue jobs. However, future scope may involve logic (possibly using AppFlow connector and Airflow Dag combination) that fetches the CSV files from SFTP and places them on S3.

The CSV files are processed sequentially within the raw layer Glue job and diagnostic information such as missing rows or empty CSV files are logged using CloudWatch logs. The CSV files are processed by a generic raw layer Glue job that uses metadata configuration to get contextual information of the CSV files. The generic Glue job is designed to handle any other datasources that generate CSV files similar to Oraclegbs. The curation layer Glue job is also generic in nature and handles the transformations using the metadata configuration of the curation layer.

6.2.1 Metadata files

The table below explains the purpose of some the important parameters.

Filename	Description
`sampling_fraction and sampling_seed`	These parameters are used to collect a sample set of data which is further used to detect schema changes.
`replace_non_alphanumeric_with_underscore`	If the raw layer columns contain non alphanumeric characters they will be replaced by an underscore `_`.
`is_fixed_file_name, file_prefix_length, file_suffix_length`	In some cases the file name may end with varying date and time stamps. In order to generically identify and parse files these parameters are used.
`separator`	The CSV files can have various separators such as comma-separated, tab-separated, or pipe-separated.

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```
---
---
SourceSystemId: oraclegbs
MetadataType: csv#oraclegbs
aws_region: ap-southeast-2
Env: dev
is_active: 'y'
job_parameter:
  bucket_name: <raw-layer-bucket-name>
  csv_files_path: oracle_gbs/csv/
  drop_duplicate_columns: true
  drop_duplicate_rows: true
  parquet_files_path: oracle_gbs/parquet/
  raw_files_path: oracle_gbs/raw/
  replacement_char: _
  replace_non_alphanumeric_with_underscore: true
  replace_non_printable_ascii_with_underscore: false
  sample_data_location: oracle_gbs/data_sampling
  sampling_fraction: '0.5'
  sampling_seed: 42
  specialchars_to_be_replaced_in_columnnames: []
  csv_files:
    - filename: EVA_Currency_Exchange_Rate_20240610_0229.CSV
      is_fixed_file_name: true
      file_prefix_length: 0
      file_suffix_length: 0
      multiline: true
      quote_character: '"'
      separator: ','
      tablename: currency_exchange_rate
      with_header: true
    - filename: Project_Customer_List_20210425090217.csv
      is_fixed_file_name: true
      file_prefix_length: 0
      file_suffix_length: 0
      multiline: true
```

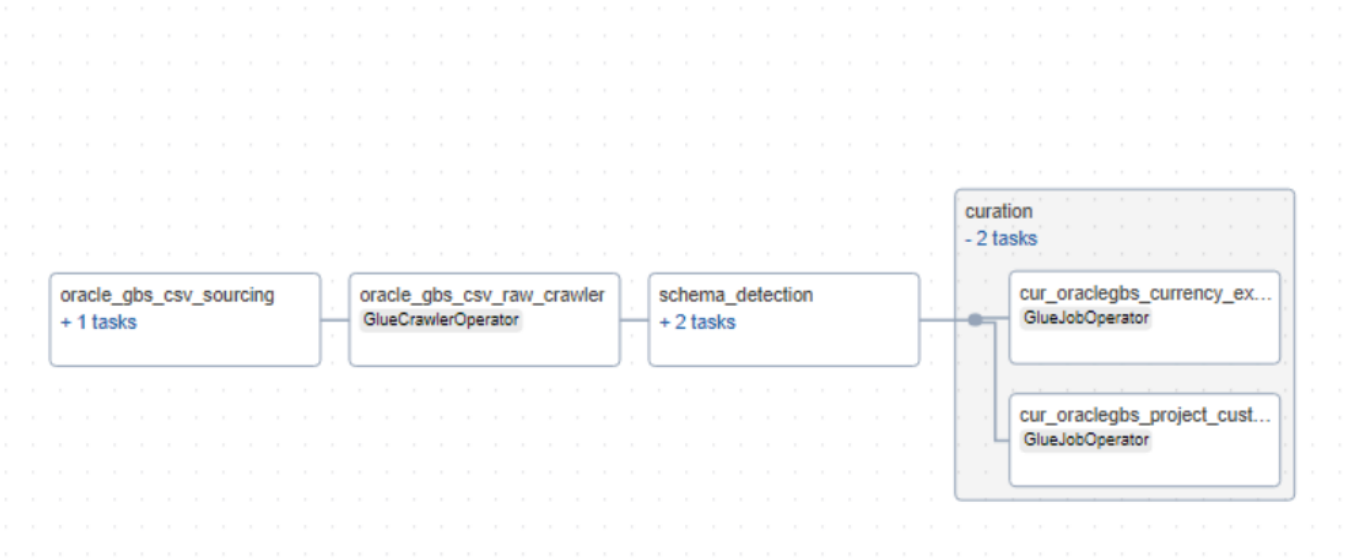
```
      quote_character: '"'
      separator: '|'
      tablename: project_customers
      with_header: true
name: oracle_gbs
Namespace: global
```

Curation Layer Metadata Example

```
---
SourceSystemId: oraclegbs_curated
MetadataType: curated#oraclegbs#currency_exchange_rate#job#iceberg
source:
  compute_engine: spark
  glue_options:
    connection_options: s3://<raw-layer-bucket-name>/oracle_gbs/parquet/currency_exchange_rate.parquet/
    connection_type: s3
    format: parquet
    format_options:
      mergeSchema: true
    transformation_ctx: currency_exchange_rate
  name: currency_exchange_rate
  spark_options:
    format: parquet
target:
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_oraclegbs
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: <curated-layer-bucket-name>/oracle_gbs/currency_exchange_rate/
      table_properties:
        format-version: '2'
        write.format.default: parquet
    table_name: curated_currency_exchange_rate
  load_type: incremental
  name: curated_currency_exchange_rate
  primary_key: br_key
  spark_options:
    format: iceberg
    options:
      path: s3://<curated-layer-bucket-name>/oracle_gbs/currency_exchange_rate
transforms:
- rename_column: true
  transform: rename_columns
- select_columns: true
  transform: select_columns_from_config_file
- change_types: true
  transform: change_data_types
- column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
  transform: add_run_date
- sql: SELECT datasource_num_id, br_key, gbs_key, from_currency, to_currency, conversion_type, conversion_rate, 'oracle_gbs' as SOURCE_SYSTEM_NAME,
date_format(to_date(month_start_date, 'dd/MM/yyyy'), 'yyyy/MM/dd') AS month_start_date, date_format(to_date(month_end_date, 'dd/MM/yyyy'), 'yyyy/MM/dd') AS
month_end_date FROM temp_df_static
  temp_view_name: temp_df_static
  transform: custom_sql
table_schema:
  schema_properties:
    enforce: true
    primary_key: br_key
    rename_columns: true
  columns:
  - column_name: datasource_num_id
    column_data_type: decimal(38,0)
    comment: datasource_num_id
    data_classification:
    nullable: true
    raw_column_name: datasource_num_id
  - column_name: br_key
    column_data_type: decimal(38,0)
    comment: br_key
    data_classification:
    nullable: true
    raw_column_name: br_key
  - column_name: gbs_key
    column_data_type: string
    comment: gbs_key
    data_classification:
    nullable: true
    raw_column_name: gbs_key
  - column_name: from_currency
    column_data_type: string
    comment: from_currency
    data_classification:
    nullable: true
    raw_column_name: from_currency
  - column_name: to_currency
    column_data_type: string
    comment: to_currency
    data_classification:
```

```
    nullable: true
    raw_column_name: to_currency
  - column_name: conversion_type
    column_data_type: string
    comment: conversion_type
    data_classification:
    nullable: true
    raw_column_name: conversion_type
  - column_name: month_start_date
    column_data_type: date
    comment: month_start_date
    data_classification:
    nullable: true
    raw_column_name: month_start_date
  - column_name: month_end_date
    column_data_type: date
    comment: month_end_date
    data_classification:
    nullable: true
    raw_column_name: month_end_date
  - column_name: conversion_rate
    column_data_type: decimal (38,4)
    comment: conversion_rate
    data_classification:
    nullable: true
    raw_column_name: conversion_rate
```

6.2.2 oraclegbs Pipeline



The oraclegbs pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process CSV files.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.2.3 Key Files Handling oraclegbs Files Processing

Filename	Description
`oracle_gbs_csv_pipeline.py`	The Airflow DAG pipeline for handling oraclegbs files processing.
`convert_csv_and_xlsx_to_parquet.py`	Generic Glue job that converts CSV and XLSX files to Parquet format. Expects the following gluejob input parameters: `--source_name:oraclegbs, --metadata_type:csv#oraclegbs, --function_name:csv, --metadata_table_name:<dynamodb_tablename>`
`schema_change_detection_generic.py`	Generic Glue job that detects schema changes between current and previous versions. Expects the following gluejob input parameters: `--catalog_db:<glue_catalogue_db_for_oraclegbs>, --table_name:<raw_table_name_of_the_csv_file>`
`raw_curated_generic.py`	Generic Glue job that creates iceberg tables. Expects the following gluejob input parameters: `--source_system_id: oraclegbs_curated, --metadata_type: curated#oraclegbs#<table_name_of_the_csv_file>#job#iceberg, --metadata_table_name:<dynamodb_tablename>`.

6.2.4 Process to onboard new files to oraclegbs

- Add the metadata of the new file to raw layer metadata file.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the curation layer table to the Airflow DAG `oracle_gbs_csv_pipeline.py`.

6.2.5 Future Scope

- Connector functionality to fetch CSV files from SFTP server and upload it to S3.
- Customizing raw layer metadata configuration to dynamically decide if Glue job processing should exit in the event of errors originating from one or more CSV files or gracefully catch the errors and continue processing rest of the files.

6.3 Overview of E3D

E3D is a datasource that is hosted on on-prem Windows Fileshare. The datasource consists of CSV and XLSX files. E3D is scoped to a project, such as "VG" or "DAC" etc.

Currently, the CSV and XLSX files are uploaded manually to a S3 bucket and these files are consumed by raw layer and curation layer Glue jobs. However, future scope may involve logic (possibly using AppFlow connector and Airflow Dag combination) that fetches the CSV and XLSX files and places them on S3.

The CSV and XLSX files are processed sequentially within the raw layer Glue job and diagnostic information such as missing rows or empty files are logged using CloudWatch logs. The files are processed by a generic raw layer Glue job that uses metadata configuration to get contextual information of the files. The generic Glue job is designed to handle any other datasources that generate CSV or XLSX files similar to E3D. The curation layer Glue job is also generic in nature and handles the transformations using the metadata configuration of the curation layer.

6.3.1 Metadata files

The table below explains the purpose of some the important parameters.

Filename	Description
`sampling_fraction and sampling_seed`	These parameters are used to collect a sample set of data which is further used to detect schema changes.
`replace_non_alphanumeric_with_underscore`	If the raw layer columns contain non alphanumeric characters they will be replaced by an underscore `_`.
`is_fixed_file_name, file_prefix_length, file_suffix_length`	In some cases the file name may end with varying date and time stamps. In order to generically identify and parse files these parameters are used.
`separator`	The CSV files or the sheets within XLSX can have various separators such as comma-separated, tab-separated, or pipe-separated.

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```
---
SourceSystemId: e3d
MetadataType: csv_xlsx#vg#e3d
aws_region: ap-southeast-2
Env: dev
is_active: 'y'
job_parameter:
  bucket_name: <raw_layer_bucket>
  csv_files_path: vg/e3d/csv/
  drop_duplicate_columns: true
  drop_duplicate_rows: true
  parquet_files_path: vg/e3d/parquet/
  raw_files_path: vg/e3d/raw/
  replacement_char: _
  replace_non_alphanumeric_with_underscore: true
  replace_non_printable_ascii_with_underscore: false
  sample_data_location: vg/e3d/data_sampling
  sampling_fraction: '0.5'
  sampling_seed: 42
  specialchars_to_be_replaced_in_columnnames: []
  csv_files:
    - filename: PipeCompWbsRep.csv
      is_fixed_file_name: true
      file_prefix_length: 0
      file_suffix_length: 0
      multiline: true
      quote_character: '"'
      separator: ','
      tablename: pipecompwbsrep
      with_header: true
  xls_files:
    - filename: VGL-EQUIP-18062024.xlsx
      file_prefix_length: 0
      file_suffix_length: 9
      is_fixed_file_name: false
      xls_engine_name: openpyxl
```

```
      sheets:
        - sheet_name: Sheet1
          tablename: vg_e3d_vglequip_sheet
          quote_character: '"'
          separator: ','
          multiline: true
          header_row: 0
          header_column: 0
          data_start_row: 0
          data_start_column: 0



name: e3d
Namespace: vg
```

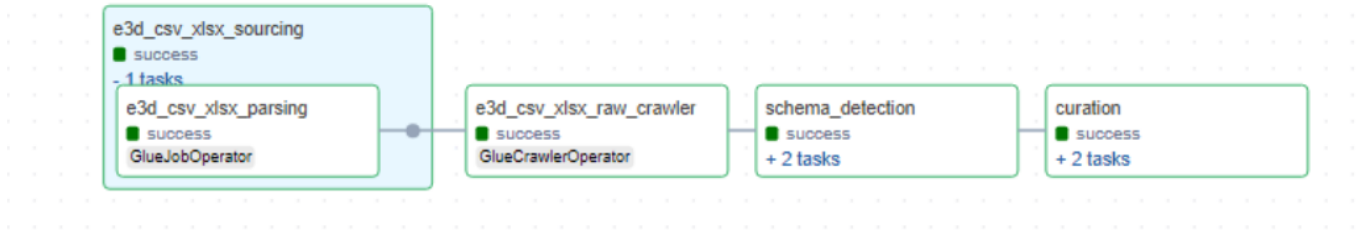
Curation Layer Metadata Example

```
---
---
SourceSystemId: e3d_curated
MetadataType: curated#vg_E3D#pipecompwbsrep#job#iceberg
source:
  compute_engine: spark
  glue_options:
    connection_options: s3://<raw_layer_bucket>/vg/e3d/parquet/pipecompwbsrep.parquet/
    connection_type: s3
    format: parquet
    transformation_ctx: vg_E3D-csv_xlsx_curated
  name: vg_E3D_csv_xlsx_curated
  spark_options:
    format: parquet
table_schema:
  columns:
  - column_data_type: string
    column_name: WBS
    comment: WBS
    data_classification:
    nullable: true
    raw_column_name: wbs
  - column_data_type: string
    column_name: SITE
    comment: SITE
    data_classification:
    nullable: true
    raw_column_name: site
  - column_data_type: string
    column_name: ZONE
    comment: ZONE
    data_classification:
    nullable: true
    raw_column_name: zone
  - column_data_type: string
    column_name: PIPE
    comment: PIPE
    data_classification:
    nullable: true
    raw_column_name: pipe
  - column_data_type: string
    column_name: BRANCH
    comment: BRANCH
    data_classification:
    nullable: true
    raw_column_name: branch
  - column_data_type: string
    column_name: TYPE
    comment: TYPE
    data_classification:
    nullable: true
    raw_column_name: type
  - column_data_type: string
    column_name: REF
    comment: REF
    data_classification:
    nullable: true
    raw_column_name: ref
  - column_data_type: string
    column_name: TAG
    comment: TAG
    data_classification:
    nullable: true
    raw_column_name: tag
  - column_data_type: string
    column_name: SPREF
    comment: SPREF
    data_classification:
    nullable: true
    raw_column_name: spref
  - column_data_type: string
    column_name: ISPEC
    comment: ISPEC
```

```
    data_classification:
    nullable: true
    raw_column_name: ispec
  - column_data_type: string
    column_name: NB1
    comment: NB1
    data_classification:
    nullable: true
    raw_column_name: nb1
  - column_data_type: string
    column_name: NB2
    comment: NB2
    data_classification:
    nullable: true
    raw_column_name: nb2
  - column_data_type: string
    column_name: NB3
    comment: NB3
    data_classification:
    nullable: true
    raw_column_name: nb3
  - column_data_type: string
    column_name: NB4
    comment: NB4
    data_classification:
    nullable: true
    raw_column_name: nb4
  - column_data_type: string
    column_name: LENGTH
    comment: LENGTH
    data_classification:
    nullable: true
    raw_column_name: length
  - column_data_type: string
    column_name: DESCRIPTION
    comment: DESCRIPTION
    data_classification:
    nullable: true
    raw_column_name: description
  - column_data_type: string
    column_name: MATERIAL
    comment: MATERIAL
    data_classification:
    nullable: true
    raw_column_name: material
  - column_data_type: string
    column_name: IDCODE
    comment: IDCODE
    data_classification:
    nullable: true
    raw_column_name: idcode
  - column_data_type: string
    column_name: CMCODE
    comment: CMCODE
    data_classification:
    nullable: true
    raw_column_name: cmcode
  - column_data_type: string
    column_name: LINENO
    comment: LINENO
    data_classification:
    nullable: true
    raw_column_name: lineno
  - column_data_type: string
    column_name: DUTY
    comment: DUTY
    data_classification:
    nullable: true
    raw_column_name: duty
  - column_data_type: string
    column_name: WGT
    comment: WGT
    data_classification:
    nullable: true
    raw_column_name: wgt_lbs
  - column_data_type: string
    column_name: WGT_LBS_W_INSU
    comment: WGT_LBS_W_INSU
    data_classification:
    nullable: true
    raw_column_name: wgt_lbs_w_insu
  - column_data_type: string
    column_name: EWP
    comment: EWP
    data_classification:
    nullable: true
    raw_column_name: ewp_ewp
  - column_data_type: string
    column_name: CWA
    comment: CWA
    data_classification:
    nullable: true
    raw_column_name: cwa_cwarea
  - column_data_type: string
```

```
    column_name: CWP
    comment: CWP
    data_classification:
    nullable: true
    raw_column_name: cwp_workpackno
  - column_data_type: string
    column_name: CWPZONE
    comment: CWPZONE
    data_classification:
    nullable: true
    raw_column_name: cwpzone_areacode
  - column_data_type: string
    column_name: ISO
    comment: ISO
    data_classification:
    nullable: true
    raw_column_name: iso
  - column_data_type: string
    column_name: DRAWING_NO
    comment: DRAWING_NO
    data_classification:
    nullable: true
    raw_column_name: drawing_no
  - column_data_type: string
    column_name: P_ID_LINE_NO
    comment: P_ID_LINE_NO
    data_classification:
    nullable: true
    raw_column_name: p_id_line_no
  - column_data_type: string
    column_name: P_ID_REFERENCE
    comment: P_ID_REFERENCE
    data_classification:
    nullable: true
    raw_column_name: p_id_reference
  schema_properties:
    enforce: true
    rename_columns: true
target:
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_vg_e3d
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: <curated_layer_bucket>/vg/e3d/pipecompwbsrep/
      table_properties:
        format-version: '2'
        write.format.default: parquet
    table_name: curated_pipecompwbsrep
  name: curated_vg_E3D_pipecompwbsrep
  load_type: append
  spark_options:
    format: iceberg
    options:
      path: s3://<curated_layer_bucket>/vg/e3d/pipecompwbsrep
transforms:
- rename_column: true
  transform: rename_columns
- select_columns: true
  transform: select_columns_from_config_file
- change_types: true
  transform: change_data_types
- column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
  transform: add_run_date
- sql: SELECT *, 'vg_E3D' as SOURCE_SYSTEM_NAME FROM temp_df_static
  temp_view_name: temp_df_static
  transform: custom_sql
```

6.3.2 E3D Pipeline



The E3D pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process CSV and XLSX files.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.3.3 Key Files Handling E3D Files Processing

Filename	Description
`e3d_csv_and_xlsx_pipeline.py`	The Airflow DAG pipeline for handling E3D files processing.
`convert_csv_and_xlsx_to_parquet.py`	Generic Glue job that converts CSV and XLSX files to Parquet format. Expects the following gluejob input parameters: `--source_name:e3d, --metadata_type:csv#e3d, --function_name:csv, --metadata_table_name:<dynamodb_tablename>`
`schema_change_detection_generic.py`	Generic Glue job that detects schema changes between current and previous versions. Expects the following gluejob input parameters: `--catalog_db:<glue_catalogue_db_for_e3d>, --table_name:<raw_table_name_of_the_csv_file>`
`raw_curated_generic.py`	Generic Glue job that creates iceberg tables. Expects the following gluejob input parameters: `--source_system_id: e3d_curated, --metadata_type: curated#e3d#<table_name_of_the_csv_file>#job#iceberg, --metadata_table_name:<dynamodb_tablename>`.

6.3.4 Process to onboard new files to E3D

- Add the metadata of the new file to raw layer metadata file.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the curation layer table to the Airflow DAG `e3d_csv_and_xlsx_pipeline.py`.

6.3.5 Future Scope

- Connector functionality to fetch CSV and XLSX files from the on-prem Windows fileshare and upload it to S3.
- Customizing raw layer metadata configuration to dynamically decide if Glue job processing should exit in the event of errors originating from one or more CSV or XLSX files or gracefully catch the errors and continue processing rest of the files.

6.4 Overview of sharepoint

Sharepoint is a datasource that hosts CSV and XLSX files on a Sharepoint platform. The datasource consists of CSV and XLSX files. Sharepoint is scoped to a project, such as "VG" or "DAC" etc.

Currently, the CSV and XLSX files are uploaded manually to a S3 bucket and these files are consumed by raw layer and curation layer Glue jobs. However, future scope may involve logic (possibly using AppFlow connector and Airflow Dag combination) that fetches the CSV and XLSX files and places them on S3.

The CSV and XLSX files are processed sequentially within the raw layer Glue job and diagnostic information such as missing rows or empty files are logged using CloudWatch logs. The files are processed by a generic raw layer Glue job that uses metadata configuration to get contextual information of the files. The generic Glue job is designed to handle any other datasources that generate CSV or XLSX files similar to sharepoint. The curation layer Glue job is also generic in nature and handles the transformations using the metadata configuration of the curation layer.

6.4.1 Metadata files

The table below explains the purpose of some the important parameters.

Filename	Description
`sampling_fraction and sampling_seed`	These parameters are used to collect a sample set of data which is further used to detect schema changes.
`replace_non_alphanumeric_with_underscore`	If the raw layer columns contain non alphanumeric characters they will be replaced by an underscore `_`.
`is_fixed_file_name, file_prefix_length, file_suffix_length`	In some cases the file name may end with varying date and time stamps. In order to generically identify and parse files these parameters are used.
`separator`	The CSV files or the sheets within XLSX can have various separators such as comma-separated, tab-separated, or pipe-separated.

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```
---
SourceSystemId: sharepoint
MetadataType: csv_xlsx#vg#sharepoint
aws_region: ap-southeast-2
Env: dev
is_active: 'y'
job_parameter:
  bucket_name: <raw_layer_bucket>
  csv_files_path: vg/sharepoint/csv/
  drop_duplicate_columns: true
  drop_duplicate_rows: true
  parquet_files_path: vg/sharepoint/parquet/
  raw_files_path: vg/sharepoint/raw/
  replacement_char: _
  replace_non_alphanumeric_with_underscore: true
  replace_non_printable_ascii_with_underscore: false
  sample_data_location: vg/sharepoint/data_sampling
  sampling_fraction: '0.5'
  sampling_seed: 42
  specialchars_to_be_replaced_in_columnnames: []
  csv_files:
    - filename: Assumptions Index.csv
      is_fixed_file_name: true
      file_prefix_length: 0
      file_suffix_length: 0
      multiline: true
      quote_character: '"'
      separator: ','
      tablename: assumptions_index
      with_header: true
  xls_files:
    - filename: LFM ISO TRACKER.xlsx
      file_prefix_length: 0
      file_suffix_length: 0
      is_fixed_file_name: true
      xls_engine_name: openpyxl
```

```
      sheets:
        - sheet_name: 'ISO Tracker sheet '
          tablename: vg_sharepoint_lfm_iso_tracker_sheet
          quote_character: '"'
          separator: ','
          multiline: true
          header_row: 4
          header_column: 0
          data_start_row: 0
          data_start_column: 0
name: sharepoint
Namespace: vg
```

Curation Layer Metadata Example

```
---
SourceSystemId: sharepoint_curated
MetadataType: curated#vg_sharepoint#assumptions_index#job#iceberg
source:
  compute_engine: spark
  glue_options:
    connection_options: s3://<raw_layer_bucket>/vg/sharepoint/parquet/assumptions_index.parquet/
    connection_type: s3
    format: parquet
    transformation_ctx: vg_sharepoint-csv_xlsx_curated
  name: vg_sharepoint_csv_xlsx_curated
  spark_options:
    format: parquet
table_schema:
  columns:
  - column_data_type: string
    column_name: ASSUMPTION_ID
    comment: ASSUMPTION_ID
    data_classification:
    nullable: true
    raw_column_name: assumption_id
  - column_data_type: string
    column_name: BASIS_FOR_ASSUMPTION
    comment: BASIS_FOR_ASSUMPTION
    data_classification:
    nullable: true
    raw_column_name: basis_for_assumption
  - column_data_type: string
    column_name: ZONE
    comment: ZONE
    data_classification:
    nullable: true
    raw_column_name: zone
  - column_data_type: string
    column_name: WBS_AREA
    comment: WBS_AREA
    data_classification:
    nullable: true
    raw_column_name: wbs_area
  - column_data_type: string
    column_name: DOCUMENT_DRAWING_NO
    comment: DOCUMENT_DRAWING_NO
    data_classification:
    nullable: true
    raw_column_name: document_drawing_no
  - column_data_type: string
    column_name: DESIGN_DELIVERABLE_DRAWING_DESCRIPTION
    comment: DESIGN_DELIVERABLE_DRAWING_DESCRIPTION
    data_classification:
    nullable: true
    raw_column_name: design_deliverable_drawing_description
  - column_data_type: string
    column_name: DESIGN_DELIVERABLE_REV
    comment: DESIGN_DELIVERABLE_REV
    data_classification:
    nullable: true
    raw_column_name: design_deliverable_rev
  - column_data_type: string
    column_name: KEY_DOCUMENT_TO_CLOSE_ASSUMPTION
    comment: KEY_DOCUMENT_TO_CLOSE_ASSUMPTION
    data_classification:
    nullable: true
    raw_column_name: key_document_to_close_assumption
  - column_data_type: string
    column_name: ASSUMPTION_ADDED_BY
    comment: ASSUMPTION_ADDED_BY
    data_classification:
    nullable: true
    raw_column_name: assumption_added_by
  - column_data_type: string
    column_name: RESPONSIBLE_ENGINEER
    comment: RESPONSIBLE_ENGINEER
    data_classification:
    nullable: true
    raw_column_name: responsible_engineer
  - column_data_type: string
```

```
    column_name: LEAD_DISCIPLINE
    comment: LEAD_DISCIPLINE
    data_classification:
    nullable: true
    raw_column_name: lead_discipline
  - column_data_type: timestamp
    column_name: DATE_ASSUMPTION_LISTED
    comment: DATE_ASSUMPTION_LISTED
    data_classification:
    nullable: true
    raw_column_name: date_assumption_listed
  - column_data_type: string
    column_name: ASSUMPTION_CONFIRMED
    comment: ASSUMPTION_CONFIRMED
    data_classification:
    nullable: true
    raw_column_name: assumption_confirmed__yes_no
  - column_data_type: timestamp
    column_name: DATE_ASSUMPTION_CONFIRMED
    comment: DATE_ASSUMPTION_CONFIRMED
    data_classification:
    nullable: true
    raw_column_name: date_assumption_confirmed
  - column_data_type: timestamp
    column_name: DATE_DESIGN_DELIVERABLE_UPDATED_AND_ISSUED
    comment: DATE_DESIGN_DELIVERABLE_UPDATED_AND_ISSUED
    data_classification:
    nullable: true
    raw_column_name: date_design_deliverable_updated_and_issued_if_assumption_is_invalid
  - column_data_type: string
    column_name: UPDATE_TO_ASSUMPTION_IF_INVALID
    comment: UPDATE_TO_ASSUMPTION_IF_INVALID
    data_classification:
    nullable: true
    raw_column_name: update_to_assumption_if_invalid
  - column_data_type: string
    column_name: STATUS
    comment: STATUS
    data_classification:
    nullable: true
    raw_column_name: status_open_closed
  - column_data_type: string
    column_name: REMARKS
    comment: REMARKS
    data_classification:
    nullable: true
    raw_column_name: remarks
  schema_properties:
    enforce: true
    rename_columns: true
target:
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_vg_sharepoint
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: <curated_layer_bucket>/vg/sharepoint/assumptions_index/
      table_properties:
        format-version: '2'
        write.format.default: parquet
    table_name: curated_assumptions_index
  name: curated_vg_sharepoint_assumptions_index
  load_type: append
  spark_options:
    format: iceberg
    options:
      path: s3://<curated_layer_bucket>/vg/sharepoint/assumptions_index
transforms:
- rename_column: true
  transform: rename_columns
- select_columns: true
  transform: select_columns_from_config_file
- change_types: true
  transform: change_data_types
- column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
  transform: add_run_date
- sql: SELECT *, 'vg_sharepoint' as SOURCE_SYSTEM_NAME FROM temp_df_static
  temp_view_name: temp_df_static
  transform: custom_sql
```

6.4.2 sharepoint Pipeline



The sharepoint pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process CSV and XLSX files.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.4.3 Key Files Handling sharepoint Files Processing

Filename	Description
`sharepoint_csv_and_xlsx_pipeline.py`	The Airflow DAG pipeline for handling sharepoint files processing.
`convert_csv_and_xlsx_to_parquet.py`	Generic Glue job that converts CSV and XLSX files to Parquet format. Expects the following gluejob input parameters: `--source_name:sharepoint, --metadata_type:csv#sharepoint, --function_name:csv, --metadata_table_name:<dynamodb_tablename>`
`schema_change_detection_generic.py`	Generic Glue job that detects schema changes between current and previous versions. Expects the following gluejob input parameters: `--catalog_db:<glue_catalogue_db_for_sharepoint>, --table_name:<raw_table_name_of_the_csv_file>`
`raw_curated_generic.py`	Generic Glue job that creates iceberg tables. Expects the following gluejob input parameters: `--source_system_id: sharepoint_curated, --metadata_type: curated#sharepoint#<table_name_of_the_csv_file>#job#iceberg, --metadata_table_name:<dynamodb_tablename>`.

6.4.4 Process to onboard new files to sharepoint

- Add the metadata of the new file to raw layer metadata file.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the curation layer table to the Airflow DAG `sharepoint_csv_and_xlsx_pipeline.py`.

6.4.5 Future Scope

- Connector functionality to fetch CSV and XLSX files from Sharepoint and upload it to S3.
- Customizing raw layer metadata configuration to dynamically decide if Glue job processing should exit in the event of errors originating from one or more CSV or XLSX files or gracefully catch the errors and continue processing rest of the files.

6.5 Overview of Ecosys

Data from Ecosys system is exposed in the form of APIs. The requirement is to extract the data from the given Ecosys APIs and load them to the Datalake tables using AWS glue jobs orchestrated via Airflow .

Ecosys has two workflows -First Workflow is a generic workflow which encompasess all the API's which is extracted at the Universal SecOrg level or without SecOrg -Second workflow encompases the Project specifc workflow eg:VG,DAC etc .

6.5.1 API's Info

The table below explains the list of API's , Workflow category, Parameter Passed , API URL , Load Type, Primary Key.

APIName	WorkflowCategiry	Parameter Passed	APIURL	LoadType	PrimaryKey
`project_list`	Project Specific	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/Ecosys_ProjectList/	Incremental	ProjectInterna
`gateData`	Project Specific	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/INTa_DEL_EDSR_Gate_Data/	fullload	No Primary K
`dcsmapping`	Project Specific	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/INTa_DEL_EDSR_DCS_Mapping/	fullload	No Primary K
`timephased`	Project Specific	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/Ecosys_ControlAccount_TimePhased/	fullload+append	No Primary K
`header`	Generic and No SecOrg	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/INTa_SnowFlake_ECOHDR_CO/	Incremental	ProjectInterna
`secorglist`	Generic and No SecOrg	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/INTa_SnowFlake_SecOrgs/	fullload	SecOrgID
`snapidlist`	Generic and No SecOrg	Source Name , Function Name and SecOrg	https://ecosys-int.worley.com/ecosys/api/restjson/INTa_SnowFlake_SnapID/	fullload	TransactionC
`snapshotfull`	Generic and SecOrg	Source Name , Function Name and SecOrg	https://ecosys.worley.com/ecosys/api/restjson/INTa_SnowFlake_Snapshot_List_SecOrg/	fullload+append	SnapshotIntI

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```
---
SourceSystemId: ecosys
MetadataType: api#ecosys#project_list
name: project_list
is_active: y
api_parameter:
  api_method: get # Mandatory (Key and Value)
  api_response_type : raw
  endpoint: https://ecosys-int.worley.com/ecosys/api/restjson/Ecosys_ProjectList/ #Mandatory (Key and Value)
  api_auth_type: Basic #Mandatory (Key and Value)
  secret_key: worley-datalake-sydney-dev-api-ecosys  # This goes into SecretsManager (Mandatory) (Key and Value)
  api_ssl_verify : False
  api_retry: 3 #Mandatory (Key and Value)
  api_timeout: 300 #Mandatory (Key and Value)
  api_headers: #Mandatory
```

```
    content_type: application/json #Mandatory (Key and Value)
  dynamic_api_headers: #Mandatory
    Authorization : None #Optional (Key only)
  api_query_params: None #Mandatory (Key only)
  dynamic_api_query_param :  #Optional
  api_body: None #Mandatory (Key only)
job_parameter:
  bucket_name: worley-datalake-sydney-dev-bucket-raw-xd5ydg
  input_path : None
  full_incr: f
  table_prefix: project_list
  secorg_id : HOUS-B2d90
```

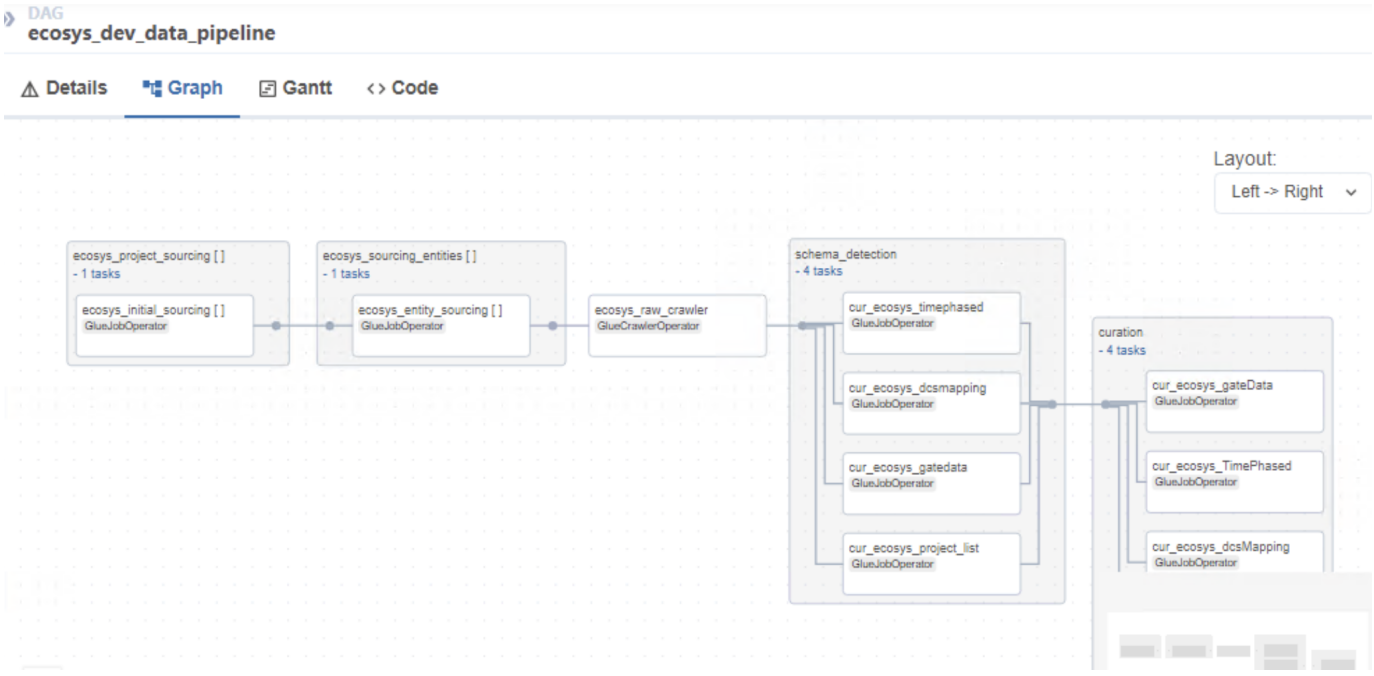
Curation Layer Metadata Example

```
---
---
SourceSystemId: ecosys_curated
MetadataType: curated#ecosys#project_list#job#iceberg
source:
  name: ecosys_project_list
  compute_engine: spark
  spark_options:
    format: parquet
  glue_options:
    connection_type: s3
    connection_options: s3://worley-datalake-sydney-dev-bucket-raw-xd5ydg/ecosys/project_list/
    format: parquet
    transformation_ctx: ecosys-project_list
transforms:
- transform: rename_columns
  rename_column: true
- transform: select_columns_from_config_file
  select_columns: true
- transform: change_data_types
  change_types: true
- transform: add_run_date
  column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
- transform: custom_sql
  sql: SELECT *, 'ecosys' as SOURCE_SYSTEM_NAME FROM temp_df_static
  temp_view_name: temp_df_static
target:
  name: curated_ecosys_project_list
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_ecosys
    table_name: curated_project_list
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/ecosys/project_list/
      table_properties:
        write.format.default: parquet
        format-version: '2'
  load_type: incremental
  spark_options:
    format: iceberg
    options:
      path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/ecosys/project_list
table_schema:
  schema_properties:
    primary_key: [ProjectInternalID]
    enforce: true
    rename_columns: true
  columns:
  - column_name: SecOrg
    column_data_type: string
    nullable: true
    comment: SecOrg
    data_classification: null
    raw_column_name: projectlist_secorg
  - column_name: Project
    column_data_type: string
    nullable: true
    comment: Project
    data_classification: null
    raw_column_name: projectlist_project
  - column_name: ProjectInternalID
    column_data_type: string
    nullable: true
    comment: Project Internal ID
    data_classification: null
    raw_column_name: projectlist_projectinternalid
  - column_name: SystemDate
    column_data_type: date
    nullable: true
    comment: System Date
    data_classification: null
    raw_column_name: projectlist_systemdate
  - column_name: LatestSnapshotDate
    column_data_type: date
```

```
    nullable: true
    comment: Latest Snapshot Date
    data_classification: null
    raw_column_name: projectlist_latestsnapshotdate
  - column_name: ProjectName
    column_data_type: string
    nullable: true
    comment: Project Name
    data_classification: null
    raw_column_name: projectlist_projectname
  - column_name: ProjectType
    column_data_type: string
    nullable: true
    comment: Project Type
    data_classification: null
    raw_column_name: projectlist_projecttype
  - column_name: Client
    column_data_type: string
    nullable: true
    comment: Client
    data_classification: null
    raw_column_name: projectlist_client
  - column_name: Scope
    column_data_type: string
    nullable: true
    comment: Scope
    data_classification: null
    raw_column_name: projectlist_scope
  - column_name: Status
    column_data_type: string
    nullable: true
    comment: Project Approval Status (ID)
    data_classification: null
    raw_column_name: projectlist_status
  - column_name: PortfolioProjectGrouping
    column_data_type: string
    nullable: true
    comment: Portfolio Project Grouping
    data_classification: null
    raw_column_name: projectlist_portfolioprojectgrouping
  - column_name: PM
    column_data_type: string
    nullable: true
    comment: Project Manager (Name)
    data_classification: null
    raw_column_name: projectlist_pm
  - column_name: PC
    column_data_type: string
    nullable: true
    comment: Project Controller (Name)
    data_classification: null
    raw_column_name: projectlist_pc
  - column_name: CustomerPM
    column_data_type: string
    nullable: true
    comment: Customer Project Manager
    data_classification: null
    raw_column_name: projectlist_customerpm
  - column_name: CustomerRep
    column_data_type: string
    nullable: true
    comment: Customer Representative
    data_classification: null
    raw_column_name: projectlist_customerrep
  - column_name: ProjectSizeClassification
    column_data_type: string
    nullable: true
    comment: Project Size Classification (Name)
    data_classification: null
    raw_column_name: projectlist_projectsizeclassification
  - column_name: ProjectApprovalStatus
    column_data_type: string
    nullable: true
    comment: Project Approval Status (Name)
    data_classification: null
    raw_column_name: projectlist_projectapprovalstatus
  - column_name: ProjectStartDate
    column_data_type: date
    nullable: true
    comment: Project Start Date
    data_classification: null
    raw_column_name: projectlist_projectstartdate
  - column_name: ProjectEndDate
    column_data_type: date
    nullable: true
    comment: Project End Date
    data_classification: null
    raw_column_name: projectlist_projectenddate
  - column_name: BillingType
    column_data_type: string
    nullable: true
    comment: Billing Type (Name)
    data_classification: null
    raw_column_name: projectlist_billingtype
```

```
  - column_name: ProjectRiskClassification
    column_data_type: string
    nullable: true
    comment: Project Risk Classification (Name)
    data_classification: null
    raw_column_name: projectlist_projectriskclassification
  - column_name: ScopeofService
    column_data_type: string
    nullable: true
    comment: Service Type (Name)
    data_classification: null
    raw_column_name: projectlist_scopeofservice
  - column_name: CountryofAsset
    column_data_type: string
    nullable: true
    comment: Country of Asset (Name)
    data_classification: null
    raw_column_name: projectlist_countryofasset
  - column_name: ProjectExecutionLocation
    column_data_type: string
    nullable: true
    comment: Project Execution Location
    data_classification: null
    raw_column_name: projectlist_projectexecutionlocation
  - column_name: AssetType
    column_data_type: string
    nullable: true
    comment: AssetType (Name)
    data_classification: null
    raw_column_name: projectlist_assettype
  - column_name: PublishedtoPowerBI
    column_data_type: string
    nullable: true
    comment: Published to PowerBI
    data_classification: null
    raw_column_name: projectlist_publishedtopowerbi
  - column_name: MajorProject
    column_data_type: string
    nullable: true
    comment: Major Project
    data_classification: null
    raw_column_name: projectlist_majorproject
  - column_name: HWPCOACTLibrary
    column_data_type: string
    nullable: true
    comment: H_WP COA & CT Library
    data_classification: null
    raw_column_name: projectlist_hwpcoactlibrary
#Comment
```

6.5.2 Ecosys Project Pipeline



The E3D pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process APi Files.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.5.3 Process to onboard new files to ecosys

- Add the metadata of the new file to raw layer metadata file.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the curation layer table to the Airflow DAG `ecosys_generic_pipeline.py`.

6.5.4 Future Scope

- TimePhased API to detect based on Different parameters
- User API related changes.
- Performance Optimization

6.6 Overview of Aconex

Aconex serves as a data source from which we retrieve data via an API, receiving responses in a XML format. We authenticate using a BasicAuth, where we take username/password from Secret Manager. This automated code takes projectID as argument, runs all the necessary API calls to process the data, and stores the results in PARQUET format.

Multiple AWS Glue jobs run in parallel for different tables, storing information in S3. Logs are recorded in CloudWatch, and metadata for these jobs is stored in DynamoDB. All paths and parameters are fully parameterized. Credentials for token generation, such as client ID and secret ID, are securely stored in AWS Secrets Manager.

6.6.1 Metadata files

Raw Layer Metadata Example

```yaml
---

---
SourceSystemId: aconex
MetadataType: api#aconex#workflow
api_parameter:
  api_body:
  api_exponential_backoff: 3
  api_headers:
    Content-Type: application/xml
  api_method: get
  api_query_params:
  api_response_type: xml
  api_retry: 3
  api_ssl_verify: False
  api_timeout: 360
  auth_type: Basic
  endpoint_prefix: https://
  endpoint_suffix: "/api/projects"
  secret_key: Worley-datalake-sydney-dev-db-aconex
aws_region: ap-southeast-2
is_active: "y"
job_parameter:
  bucket_data_source_prefix: aconex
  bucket_name: worley-datalake-sydney-dev-bucket-raw-xd5ydg
  full_incr: i
  incremental_criteria_folder_location: aconex/workflow_audit_folder
  incremental_default_date: '20200101'
  input_path: temp/aconex/workflow
  kms_key_id: 3a4345c9-617f-4e55-8a27-70814760f56a
  output_s3: aconex
  root_tag: Workflow
  row_tag: Workflow
  sampling_fraction: '0.5'
  sampling_seed: 42
  schema_output_s3: aconex/data_sampling
name: workflow
page_size: 500

---
```

Curation Layer Metadata Example

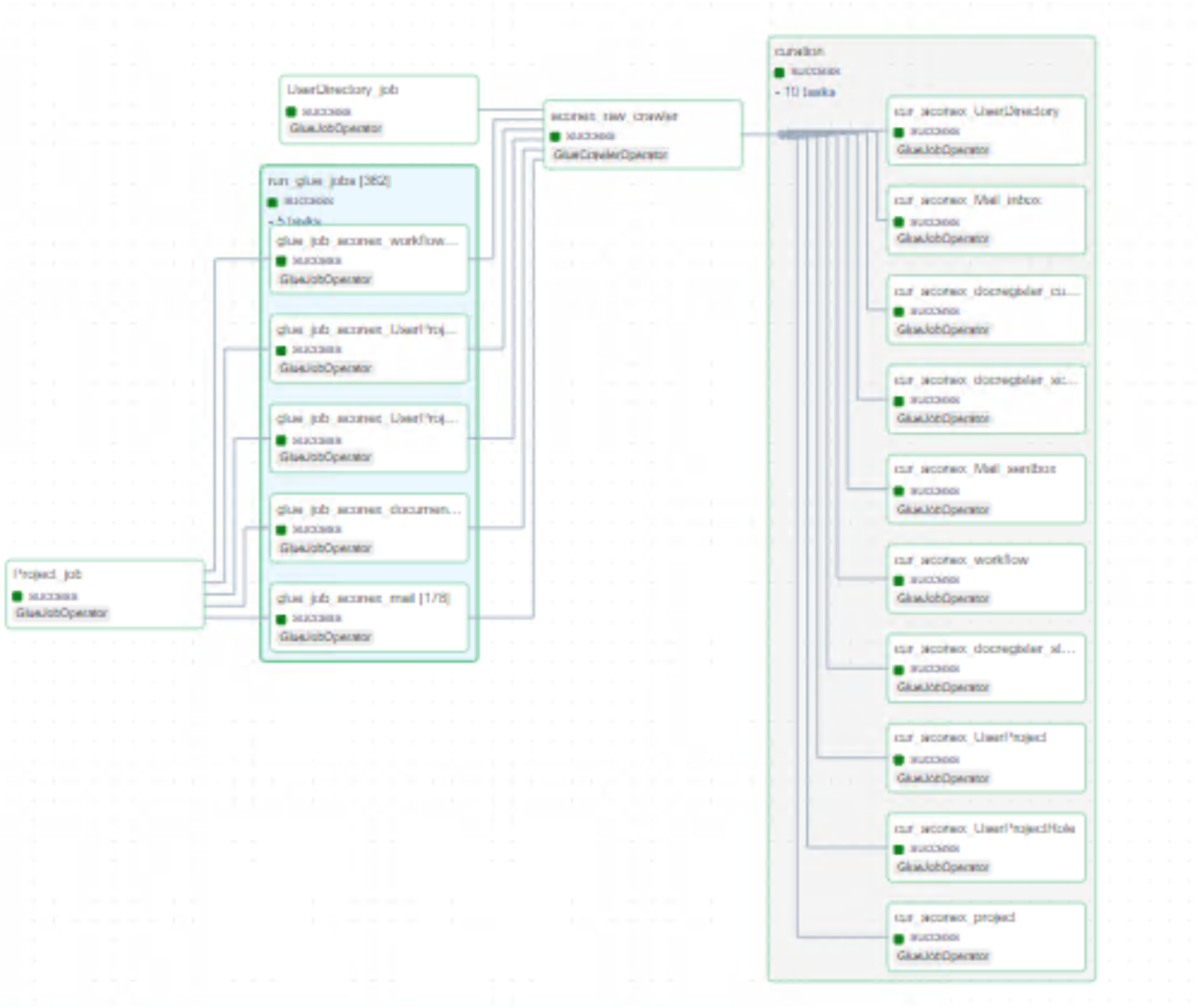
```yaml
---
---
SourceSystemId: aconex_curated
MetadataType: curated#aconex#workflow#job#iceberg
source:
  name: aconex_workflow
  compute_engine: spark
  spark_options:
    format: parquet
  glue_options:
    connection_type: s3
    connection_options: s3://worley-datalake-sydney-dev-bucket-raw-xd5ydg/aconex/workflow/
    format: parquet
    format_options:
      mergeSchema: true
    transformation_ctx: aconex-workflow
transforms:
- transform: rename_columns
  rename_column: true
- transform: select_columns_from_config_file
  select_columns: true
- transform: change_data_types
  change_types: true
```

```
- transform: add_run_date
  column_name: EXECUTION_DATE
  date_format: yyyy-MM-dd
- transform: custom_sql
  sql: SELECT *, 'aconex' as SOURCE_SYSTEM_NAME FROM temp_df_static
  temp_view_name: temp_df_static
target:
  name: curated_aconex_workflow
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_aconex
    table_name: curated_workflow
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/aconex/workflow/
      table_properties:
        write.format.default: parquet
        format-version: '2'
  load_type: incremental
  primary_key: [WorkflowId,projectid,Assignee_UserId]
  spark_options:
    format: iceberg
    options:
      path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/aconex/workflow
table_schema:
  schema_properties:
    enforce: true
    primary_key: [WorkflowId,projectid,Assignee_UserId]
    rename_columns: true
  columns:
  - column_name:  Comments
    column_data_type:   string
    nullable: true
    comment:  Comments
    data_classification: null
    raw_column_name:   comments
  - column_name:  DateCompleted
    column_data_type:   string
    nullable: true
    comment:  DateCompleted
    data_classification: null
    raw_column_name:   datecompleted
  - column_name:  DateDue
    column_data_type:   string
    nullable: true
    comment:  DateDue
    data_classification: null
    raw_column_name:   datedue
  - column_name:  DateIn
    column_data_type:   string
    nullable: true
    comment:  DateIn
    data_classification: null
    raw_column_name:   datein
  - column_name:  DaysLate
    column_data_type:   bigint
    nullable: true
    comment:  DaysLate
    data_classification: null
    raw_column_name:   dayslate
  - column_name:  DocumentNumber
    column_data_type:   string
    nullable: true
    comment:  DocumentNumber
    data_classification: null
    raw_column_name:   documentnumber
  - column_name:  DocumentTitle
    column_data_type:   string
    nullable: true
    comment:  DocumentTitle
    data_classification: null
    raw_column_name:   documenttitle
  - column_name:  DocumentTrackingId
    column_data_type:   long
    nullable: true
    comment:  DocumentTrackingId
    data_classification: null
    raw_column_name:   documenttrackingid
  - column_name:  DocumentVersion
    column_data_type:   long
    nullable: true
    comment:  DocumentVersion
    data_classification: null
    raw_column_name:   documentversion
  - column_name:  Duration
    column_data_type:   float
    nullable: true
    comment:  Duration
    data_classification: null
    raw_column_name:   duration
  - column_name:  FileName
    column_data_type:   string
    nullable: true
    comment:  FileName
```

```
    data_classification: null
    raw_column_name:   filename
  - column_name:  FileSize
    column_data_type:   long
    nullable: true
    comment:  FileSize
    data_classification: null
    raw_column_name:   filesize
  - column_name:  Initiator_Name
    column_data_type:   string
    nullable: true
    comment:  Initiator_Name
    data_classification: null
    raw_column_name:   initiator_name
  - column_name:  Initiator_OrganizationId
    column_data_type:   long
    nullable: true
    comment:  Initiator_OrganizationId
    data_classification: null
    raw_column_name:   initiator_organizationid
  - column_name:  Initiator_OrganizationName
    column_data_type:   string
    nullable: true
    comment:  Initiator_OrganizationName
    data_classification: null
    raw_column_name:   initiator_organizationname
  - column_name:  Initiator_UserId
    column_data_type:   long
    nullable: true
    comment:  Initiator_UserId
    data_classification: null
    raw_column_name:   initiator_userid
  - column_name:  OriginalDueDate
    column_data_type:   string
    nullable: true
    comment:  OriginalDueDate
    data_classification: null
    raw_column_name:   originalduedate
  - column_name:  ReasonForIssue
    column_data_type:   string
    nullable: true
    comment:  ReasonForIssue
    data_classification: null
    raw_column_name:   reasonforissue
  - column_name:  RelatedSupplierDocumentPackage
    column_data_type:   string
    nullable: true
    comment:  RelatedSupplierDocumentPackage
    data_classification: null
    raw_column_name:   relatedsupplierdocumentpackage
  - column_name:  Reviewer_Name
    column_data_type:   string
    nullable: true
    comment:  Reviewer_Name
    data_classification: null
    raw_column_name:   reviewer_name
  - column_name:  Reviewer_OrganizationId
    column_data_type:   long
    nullable: true
    comment:  Reviewer_OrganizationId
    data_classification: null
    raw_column_name:   reviewer_organizationid
  - column_name:  Reviewer_OrganizationName
    column_data_type:   string
    nullable: true
    comment:  Reviewer_OrganizationName
    data_classification: null
    raw_column_name:   reviewer_organizationname
  - column_name:  Reviewer_UserId
    column_data_type:   long
    nullable: true
    comment:  Reviewer_UserId
    data_classification: null
    raw_column_name:   reviewer_userid
  - column_name:  StepName
    column_data_type:   string
    nullable: true
    comment:  StepName
    data_classification: null
    raw_column_name:   stepname
  - column_name:  StepOutcome
    column_data_type:   string
    nullable: true
    comment:  StepOutcome
    data_classification: null
    raw_column_name:   stepoutcome
  - column_name:  StepStatus
    column_data_type:   string
    nullable: true
    comment:  StepStatus
    data_classification: null
    raw_column_name:   stepstatus
  - column_name:  WorkflowName
    column_data_type:   string
```

```
    nullable: true
    comment:  WorkflowName
    data_classification: null
    raw_column_name:   workflowname
  - column_name:  WorkflowNumber
    column_data_type:   string
    nullable: true
    comment:  WorkflowNumber
    data_classification: null
    raw_column_name:   workflownumber
  - column_name:  WorkflowStatus
    column_data_type:   string
    nullable: true
    comment:  WorkflowStatus
    data_classification: null
    raw_column_name:   workflowstatus
  - column_name:  WorkflowTemplate
    column_data_type:   string
    nullable: true
    comment:  WorkflowTemplate
    data_classification: null
    raw_column_name:   workflowtemplate
  - column_name:  WorkflowId
    column_data_type:   long
    nullable: true
    comment:  WorkflowId
    data_classification: null
    raw_column_name:   _workflowid
  - column_name:  ParentWorkflowId
    column_data_type:   long
    nullable: true
    comment:  ParentWorkflowId
    data_classification: null
    raw_column_name:   parentworkflowid
  - column_name:  Assignee_Name
    column_data_type:   string
    nullable: true
    comment:  Assignee_Name
    data_classification: null
    raw_column_name:   assignee_name
  - column_name:  Assignee_OrganizationName
    column_data_type:   string
    nullable: true
    comment:  Assignee_OrganizationName
    data_classification: null
    raw_column_name:   assignee_organizationname
  - column_name:  Assignee_OrganizationId
    column_data_type:   long
    nullable: true
    comment:  Assignee_OrganizationId
    data_classification: null
    raw_column_name:   assignee_organizationid
  - column_name:  Assignee_UserId
    column_data_type:   long
    nullable: true
    comment:  Assignee_UserId
    data_classification: null
    raw_column_name:   assignee_userid
  - column_name:  projectid
    column_data_type:   long
    nullable: true
    comment:  projectid
    data_classification: null
    raw_column_name:   projectid
  - column_name:  DocumentRevision
    column_data_type:   string
    nullable: true
    comment:  DocumentRevision
    data_classification: null
    raw_column_name:   documentrevision
```

6.6.2 Aconex Pipeline



The aconex pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process API data.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.6.3 Process to onboard new tables to Aconex

- Create metadata for that table. Run the Glue job by passing the project ID to it.
- Add a new metadata file in curation layer defining raw layer to curation layer column mappings.
- Add the table name to the Airflow DAG `aconex_pipeline.py`.

6.7 Overview of Oracle P6

Oracle P6 is an API based datasource. Oracle P6 has 3 API Endpoints: Extract API, Activity Spread API and the Resource Spread API. All the 3 APIs are Auth protected. The OAuth endpoint expects a base64 encoded username and password that is stored in the Secrets Manager. This operation is ClickOps and is not part of Terraform automation. The OAuth endpoint returns a token which is then used by the 3 APIs. The Oracle P6 endpoint is scoped to different projects, such as VG, DAC, etc. The Extract API returns XML payload and the Spread APIs return JSON payload.

The Extract API is a HTTP PUT API and expects a JSON body payload. The payload is maintained on a S3 bucket: `<bucketname>/config/oracle_p6/P6ProjectExportInput.json`. The payload needs to be uploaded to the S3 bucket via ClickOps.

Each project (such as projects in VG or DAC) can have multiple Project Ids. The orchestration of project Ids is managed by Airflow DAG. The Project Ids are mantained in a dynamodb table (also referred to as metadata framework).

The Oracle P6 Airflow DAG initially invokes the Extract API Gluejob that parses the XML payload and tokenizes various nodes within the XML payload. Each node translates to an entity (also a table in the Glue Catalogue). Project, ActivityCode etc are some examples of the XML nodes. The XML nodes that need to be parsed and converted to Parquet format are defined in the raw layer metadata configuration. Once the Extract API is executed successfully, the DAG trigers the Spread API GlueJob (Activity and Resource Assignment) concurrently. These Spread APIs receive JSON payload as input and the GlueJob converts them to Parquet files. In the current implementation, any failures or timeouts arised out of the Oracle P6 API endpoints are retried by the GlueJob retry settings.

6.7.1 Metadata files

The table below explains the purpose of some the important parameters.

Attribute name	Description
`sampling_fraction and sampling_seed`	These parameters are used to collect a sample set of data which is further used to detect schema changes.
`source_tables renamed_tables`	Indicates the nodes within the XML payload of Extract API that needs to be parsed and the corresponding table naming convention for such nodes.
`input_path`	The location of the GZIPPED XML data of the Extract API.
`schema_output_s3`	The location of sampled data on S3 for each of the sampled XML nodes.

Below is a sample raw layer metadata file.

Raw Layer Metadata Example

```
---
---
SourceSystemId: oracle_p6
MetadataType: api#oracle_p6#extract_api
name: Oracle_P6_ExtractAPI
aws_region: ap-southeast-2
is_active: "y"
auth_api_parameter:
  auth_method: post
  endpoint: https://p6.oraclecloud.com/worley/p6ws/oauth/token
  secret_key: Worley-datalake-sydney-dev-db-p6
  auth_retry: 3
  auth_timeout: 600
  auth_exponential_backoff: 5
  auth_headers:
    Content-Type: application/json
    token_exp: '3600'
  dynamic_auth_headers:
  - AuthToken
  auth_query_params:
  auth_body:
api_parameter:
  api_method: post
  endpoint: https://p6.oraclecloud.com/worley/p6ws/restapi/export/exportProject
  api_response_type: binary
  api_retry: 3
  api_timeout: 360
  api_exponential_backoff: 3
  auth_type: Bearer
  api_headers:
```

```
    Content-Type: application/json
  dynamic_api_headers:
  - Authorization
  api_query_params:
  dynamic_api_query_params:
  api_body:
  custom_config:
    export_api_input_body: config/oracle_p6/P6ProjectExportInput.json
job_parameter:
  bucket_name: worley-datalake-sydney-dev-bucket-raw-xd5ydg
  bucket_data_source_prefix: oracle_p6
  kms_key_id: 3a4345c9-617f-4e55-8a27-70814760f56a
  input_path: oracle_p6/temp/exportapi/project_gzip_data/
  temp_output_path: s3://worley-datalake-sydney-dev-bucket-raw-xd5ydg/oracle_p6/temp/exportapi/relationalized_data/
  output_s3: oracle_p6
  schema_output_s3: "/oracle_p6/data_sampling/"
  export_api_root_node: APIBusinessObjects
  sampling_fraction: '0.5'
  sampling_seed: 42
  activity_column: root_Project.Activity
  activity_code_column: root_Project.Activity.val.Code
  activity_code_join_condition: id
  activity_join_condition: index
  activitycodeassignment_table_name: activitycodeassignment
  full_incr: i/f
  source_tables:
  - root
  - root_Project.Activity
  - root_ActivityCode
  - root_ActivityCodeType
  - root_Project.ResourceAssignment
  - root_Project.ActivityCode
  - root_Resource
  - root_EPS
  - root_Project.ActivityCodeType
  renamed_tables:
  - project
  - project_activity
  - activitycode
  - activitycodetype
  - project_resourceassignment
  - project_activitycode
  - resource
  - eps
  - project_activitycodetype
```

Curation Layer Metadata Example

```
---
SourceSystemId: curated_oracle_p6
MetadataType: curated#oracle_p6#activitycode#job#iceberg
source:
  name: oracle_p6_activitycode
  compute_engine: spark
  spark_options:
    format: parquet
  glue_options:
    connection_type: s3
    connection_options: s3://worley-datalake-sydney-dev-bucket-raw-xd5ydg/oracle_p6/activitycode/
    format: parquet
    transformation_ctx: oracle_p6-activitycode
transforms:
  - transform: rename_columns
    rename_column: true
  - transform: select_columns_from_config_file
    select_columns: true
  - transform: change_data_types
    change_types: true
  - transform: add_run_date
    column_name: EXECUTION_DATE
    date_format: yyyy-MM-dd
  - transform: custom_sql
    sql: SELECT *, 'oracle_p6' as SOURCE_SYSTEM_NAME FROM temp_df_static
    temp_view_name: temp_df_static
target:
  name: curated_oracle_p6_activitycode
  compute_engine: spark
  iceberg_properties:
    database_name: worley_datalake_sydney_dev_glue_catalog_database_oracle_p6
    table_name: curated_activitycode
    iceberg_configuration:
      create_table: true
      iceberg_catalog_warehouse: worley-datalake-sydney-dev-bucket-curated-xd5ydg/oracle_p6/activitycode/
      table_properties:
        write.format.default: parquet
        format-version: '2'
  load_type: incremental
  spark_options:
    format: iceberg
    options:
      path: s3://worley-datalake-sydney-dev-bucket-curated-xd5ydg/oracle_p6/activitycode
```

```yaml
table_schema:
  schema_properties:
    enforce: true
    rename_columns: true
  columns:
    - column_name: CODECONCATNAME
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_codeconcatname
    - column_name: CODETYPENAME
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_codetypename
    - column_name: CODETYPEOBJECTID
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_codetypeobjectid
    - column_name: CODETYPESCOPE
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_codetypescope
    - column_name: CODEVALUE
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_codevalue
    - column_name: COLOR
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_color
    - column_name: CREATEDATE
      column_data_type: timestamp
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_createdate
    - column_name: CREATEUSER
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_createuser
    - column_name: DESCRIPTION
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_description
    - column_name: LASTUPDATEDATE
      column_data_type: timestamp
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_lastupdatedate
    - column_name: LASTUPDATEUSER
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_lastupdateuser
    - column_name: OBJECTID
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_objectid
    - column_name: PARENTOBJECTID
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_parentobjectid
    - column_name: PROJECTOBJECTID
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
      raw_column_name: activitycode_val_projectobjectid__xsi_nil
    - column_name: SEQUENCENUMBER
      column_data_type: string
      nullable: true
      comment: null
      data_classification: null
```

```
    raw_column_name: activitycode_val_sequencenumber
  - column_name:  projectid
    column_data_type:   long
    nullable: true
    comment:  projectid
    data_classification: null
    raw_column_name:   projectid
```

6.7.2 Oracle P6 Pipeline

The Oracle P6 pipeline consists of 3 important tasks:

- Triggers the raw layer Glue job to process Extract API.
- Triggers the raw layer Glue job to process Activity Spread API and Resource Spread API.
- Triggers the schema change detection Glue job to compare schema between current and previous versions.
- Triggers the curation layer Glue job that creates iceberg tables.

6.7.3 Future Scope

- The JSON payload used by Extract API (HTTP PUT Body) can be uploaded to the S3 bucket as part of CICD pipeline automation.

7. Data governance documnetation

7.1 High level design

7.1.1 Security Requirements

1. All user access to data (both modelled and raw data) will be via Redshift to provide a single point of entry.
2. Row-level security - Access on platform and via analytics is restricted based on source application user access
3. Row-level security - Access on platform and via analytics is restricted based on individual (report specific) security groups maintained manually by the business.
4. Row-level security - Access on platform and via analytics is restricted based on Excel maintained manually by data owners/approvers and ingested in central db repo.
5. Row-level security - Analytics/output can be published to external users and maintain individual user/group row-level security
6. Column-level security - Sensitive data is obfuscated (e.g. masked) on platform and via analytics and only visible to authorized users. Preferably using Format Preserving Encryption to ensure sensitive data is protected at all times and only decrypted at consumption.
7. Sensitive project data (e.g. DAC) can be containerized by project to segregate from other data on platform
8. System-to-system integrations for data sync between data platform and end applications (not users)
9. End-to-end encryption to protect data at rest and in transit
10. Separation of duties, security policies are maintained outside platform
11. Provide detailed logging and auditing capabilities for monitoring fine-grained access control activities, user behaviour analytics (UBA), ensuring compliance with organizational policies and regulatory requirements.
12. Provide pro-active user monitoring to block/alert activities.
13. Soft delete functionality to enable 'right to be forgotten'
14. Data blurring - ensuring individuals can't be identified where a small number of records are queried
15. potential requirement for row/cell level masking for sensitive project records (e.g. DAC)

7.1.2 Use Cases - Redshift & QuickSight

Phase 1 - Object Level Security

1) As data consumer or business user I want to access to source aligned tables based on AD Group membership via Redshift Query Editor V2 / QuickSight.

Phase 2 - Row and Column Level Security

2) As data consumer or business user I want to access to DAC data based on AD Group membership via Redshift Query Editor V2 / QuickSight.

7.1.3 Use Cases - Data Lake Access

Phase 1 - Object Level Security

1) Data Engineer 1 can query source aligned (P6, EcoSys, etc...) tables which are cataloged on Glue and governed via Lake Formation using Athena.

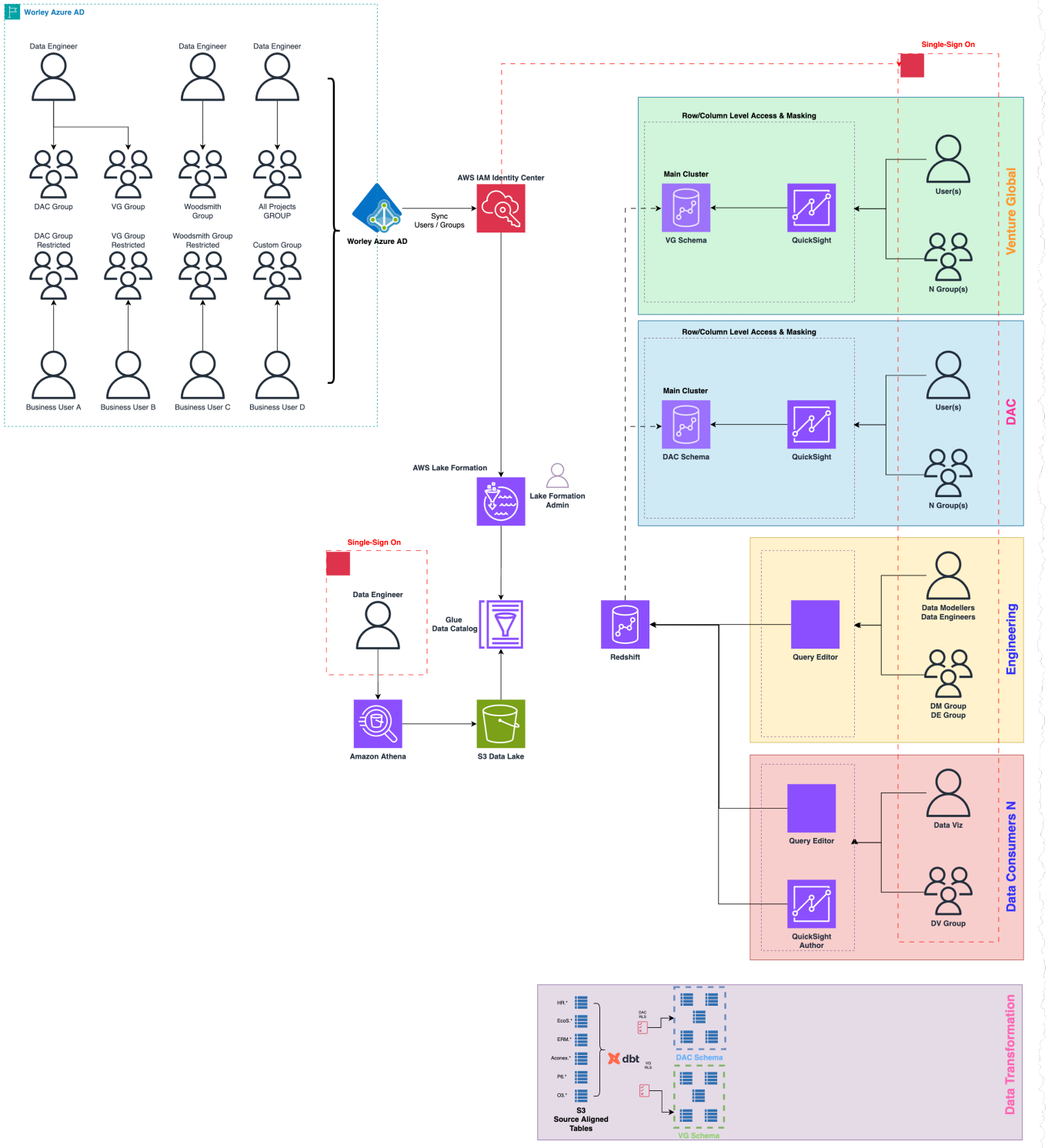
2) Data Engineer 2 cannot query source aligned (P6, EcoSys, etc...) tables which are cataloged on Glue and governed via Lake Formation using Athena.

Phase 2 - Row and Column Level Security

1) Data Engineer 1 can query DAC data which are cataloged on Glue and governed via Lake Formation using Athena.

2) Data Engineer 2 cannot query DAC data which are cataloged on Glue and governed via Lake Formation using Athena

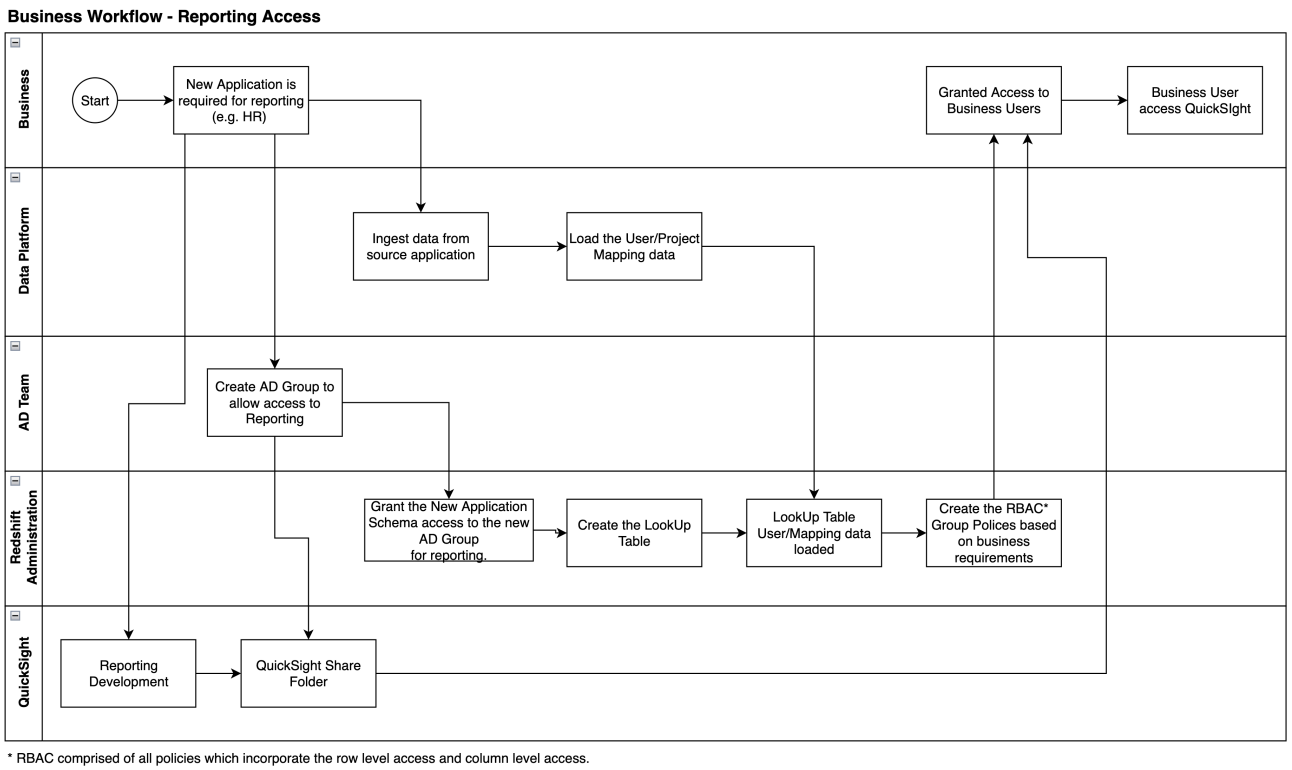
7.1.4 High-level design



7.1.5 AD Group requirements

Role	Description	Persona
Lake formation admin	The Role responsible for `granting` / `revoking` data lake permissions	data steward
Redshift DBA	The Role responsible for `granting` / `revoking` data warehouse permissions including `rls/cls/ddm` admin of policies	data steward
${source_aligned} _data_source_access	AD Group who is authorized to access the data source	data consumer
${data_product}_access	AD Group who is authorized to access data product i.e. dac-schema	business user

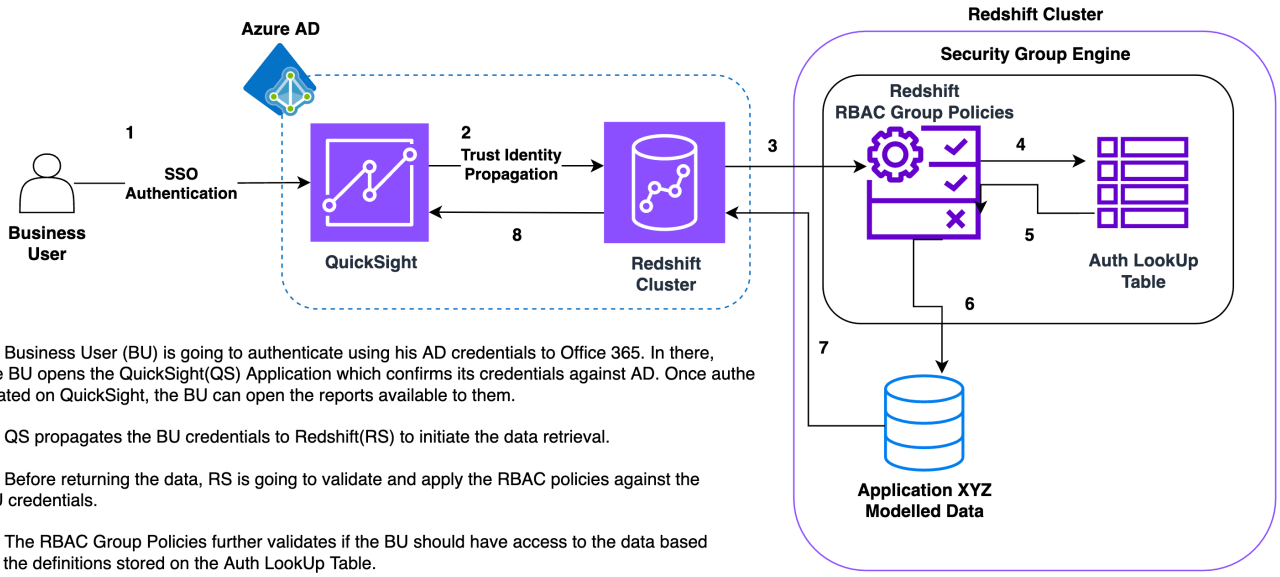
Business workflow for Reporting access via Redshift



* RBAC comprised of all policies which incorporate the row level access and column level access.

Technical Workflow - Redshift

Technical Workflow



1 - Business User (BU) is going to authenticate using his AD credentials to Office 365. In there, the BU opens the QuickSight(QS) Application which confirms its credentials against AD. Once authe ticated on QuickSight, the BU can open the reports available to them.

2 - QS propagates the BU credentials to Redshift(RS) to initiate the data retrieval.

3 - Before returning the data, RS is going to validate and apply the RBAC policies against the BU credentials.

4 - The RBAC Group Policies further validates if the BU should have access to the data based on the definitions stored on the Auth LookUp Table.

5 - The Auth LookUp Table contains the rules defined by the business which dictates what data each BU have permission to access.

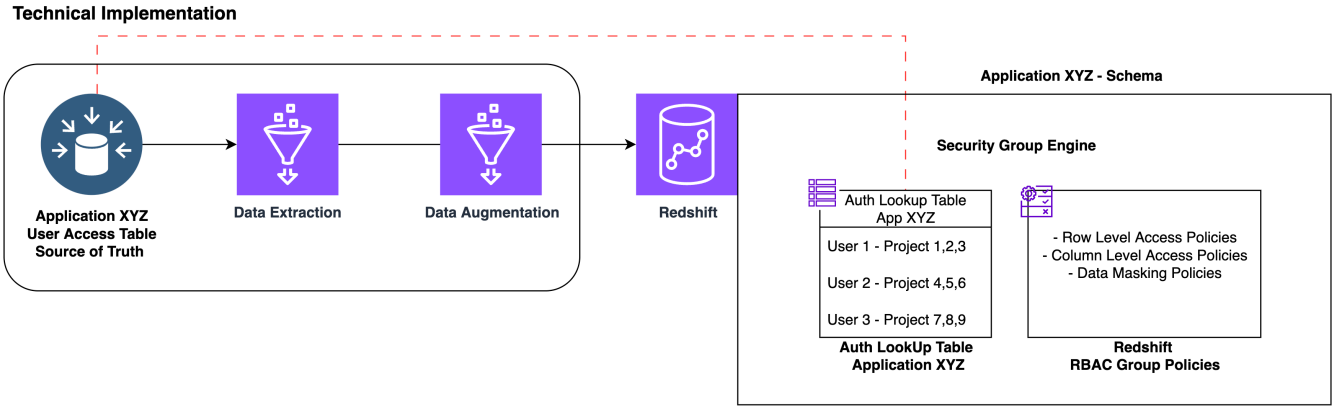
6 - Once permissions from the Auth LookUp Table are processed by the RBAC Group Policy, RS fetches only the data that the BU is authorised to visualise.

7 - Only the data that the BU is authorised to consume is returned from RS to QS.

8 - QS materialises the reports and visuals back to the BU based on the data provided by Redshift

Technical Implementation - Redshift

Technical Implementation



7.2 Redshift Spectrum Table (External Table)

This file documents the steps to create and set up Redshift Spectrum table access, along with its limitations.

7.2.1 Create Redshift Spectrum Table (External Table)

Follow these steps to create and set up Redshift Spectrum table access: 1. Sign in to the Redshift console using the admin role 2. Navigate to Query Editor v2 3. Choose the options menu(three dots) next to the cluster and choose **Create connection** 4. Connect as the super admin user and run the following commands to create an external schema from awsdatacatalog and grant access to your Identity Center group:

```
create external schema if not exists <local_schema_name> from DATA CATALOG database <federated_database_name> catalog_id <account_id>
IAM_ROLE <role_arn>

grant usage on schema <schemaname> to role <idcnamespace:rolename>;
```

For example:

```
create external schema if not exists test from DATA CATALOG database 'worley_datalake_sydney_dev_glue_catalog_database_aconex' catalog_id '891377181979'
IAM_ROLE 'arn:aws:iam::891377181979:role/worley-datalake-sydney-dev-iam-service-role-redshift-awsidc'

grant usage on schema test to role "AWSIDC:Test_Data_Consumer"
```

7.2.2 Applying RLS on external tables

⚠ Important:

Some of the columns in the external tables have data type of "string". When creating a view from these external tables, we need to change the data type of these columns to a data type that is valid in Redshift. For example, we can alter it to be "int" or "varchar".

If we don't alter the "string" data type, it will error out with "ERROR: incompatible RLS configuration on relation xxxx"

⚠ Important:

We will need to create a new schema to store these views. That is, instead of storing the tables in the source schema, we will create a new schema to store these views.

If we store the views in the same schema as the source schema, it will error out with "ERROR: Operations on local objects in external schema are not enabled"

This is an example of how you should create a schema to store the views:

```
-- Create a Schema to Store the Views
create schema test_vws_source_aconex

-- Create a View on the TABLE
create view test_vws_source_aconex.vw_curated_project as select
project::int,
projectname,
projectdescription,
execution_date,
source_system_name,
row_hash,
primary_key,
is_current,
eff_start_date,
eff_end_date
from test_source_aconex.curated_project with no schema binding

-- Grant Permissions
grant usage on schema test_vws_source_aconex to role "AWSIDC:Test_Data_Consumer"
grant select on table test_vws_source_aconex.vw_curated_project to role "AWSIDC:Test_Data_Consumer"

-- Create RLS policy
CREATE RLS POLICY show_project_1207977091_test
```

```
WITH (project int)
USING (project = '1207977091');


-- Attach Policy
ATTACH RLS POLICY show_project_1207977091_test
ON test_vws_source_aconex.vw_curated_project
TO role "AWSIDC:Test_Data_Consumer"


-- Enable RLS
ALTER table test_vws_source_aconex.vw_curated_project ROW LEVEL security on
```

⚠️ Important:

7.2.3 Limitations

Table Level Permissions Not Supported

Permissions for redshift external tables can only be managed at the schema level; it's not possible to grant permissions for specific tables individually.

For example, if you run the following query:

```
grant usage on schema test to role "AWSIDC:Test_Data_Consumer"

grant select on table test.p6_activity to role "AWSIDC:Test_Data_Consumer"
```

It will return an error message of "operation not supported on external table". This is because external tables can only be managed at the schema level.

7.3 RLS and DDL Support for External Tables in Amazon Redshift

Amazon Redshift does not currently support applying row-level security (RLS) policies or data definition language (DDL) operations directly on external tables.

7.3.1 Key Points

- You cannot attach RLS policies directly to external tables. RLS policies can only be attached to regular Redshift tables.
- DDL operations like CREATE, ALTER, DROP are supported on external schemas, but not on the individual external tables within those schemas.
- To control access to external tables, you need to grant privileges at the external schema level using GRANT commands with the IAM_ROLE option.
- For finer-grained row-level access control on data in external tables, you would need to use AWS Lake Formation permissions instead of Redshift RLS policies.

7.3.2 Summary

While RLS and full DDL support is available for regular Redshift tables, these capabilities are limited for external tables which pull data from data lakes or other external sources. The workaround is to use Lake Formation permissions and grant privileges at the external schema level in Redshift.

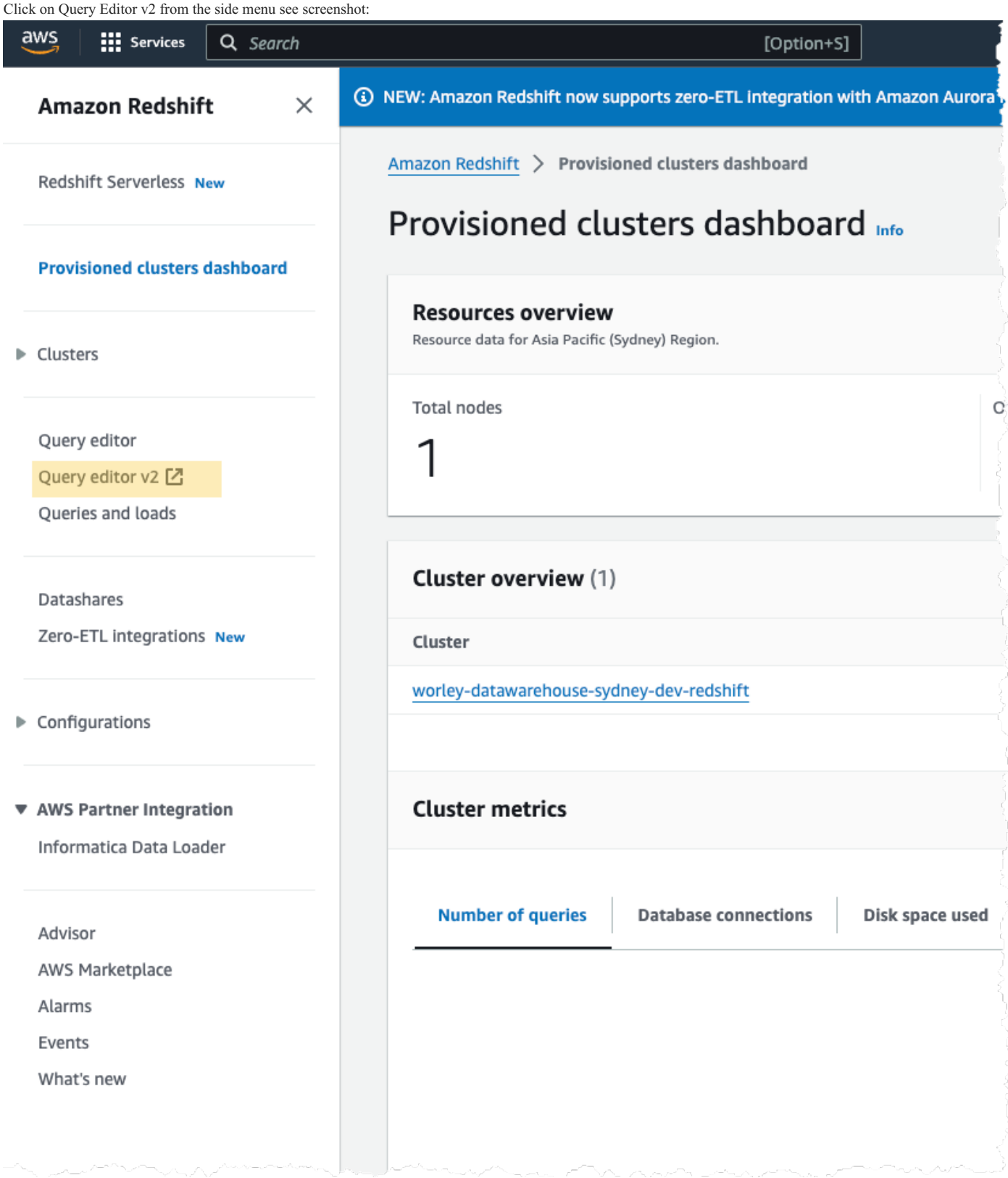
7.4 Redshift DB Admin playbook

This document outlines useful redshift admin tasks, that could be use to grant access to data users (i.e. data engineer)

7.5 Accessing the platform

1. Log onto to AWS access portal in the dev environment https://d-9767447864.awsapps.com/start/#/?tab=accounts
2. Select `RedshiftDBAdminAccess`
3. Type redshift on the search bar and go to the redshift

4. Click on Query Editor v2 from the side menu see screenshot:

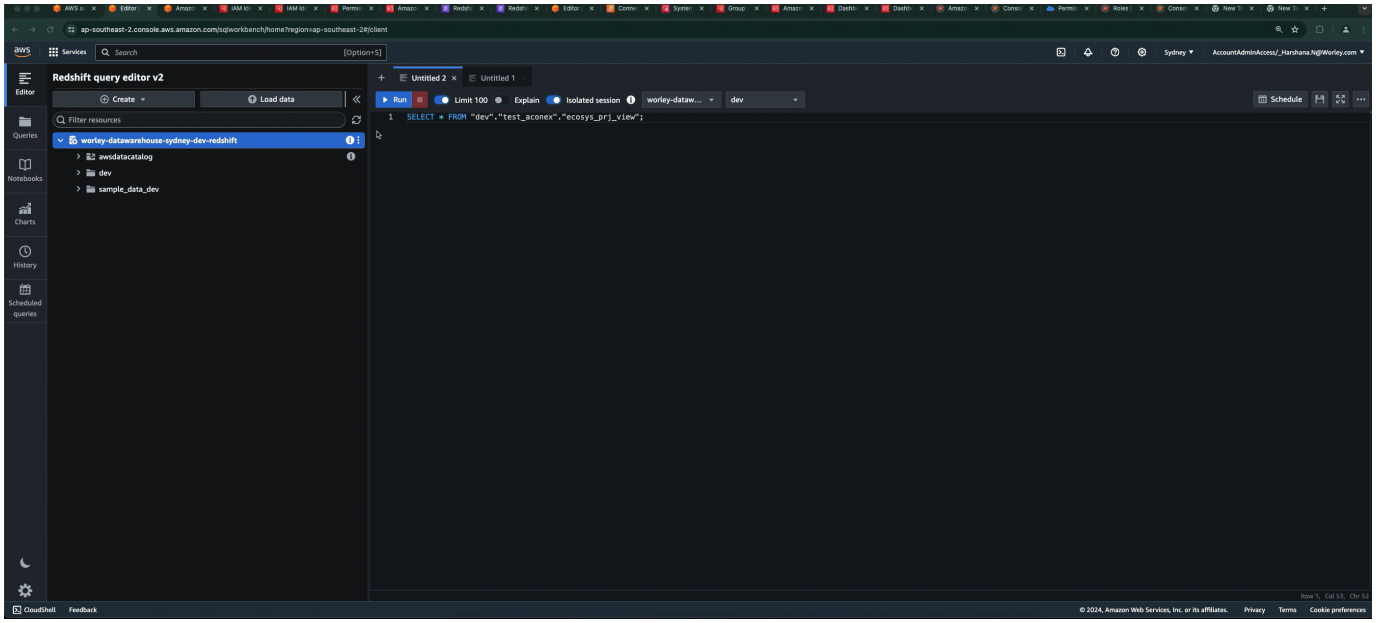


7.6 Switching Identity types in Redshift Query Editor

Certain database administration activities requires switching to different identity types e.g. Granting permissions is a super user task. this section describes how to switch identity types in redshift query editor

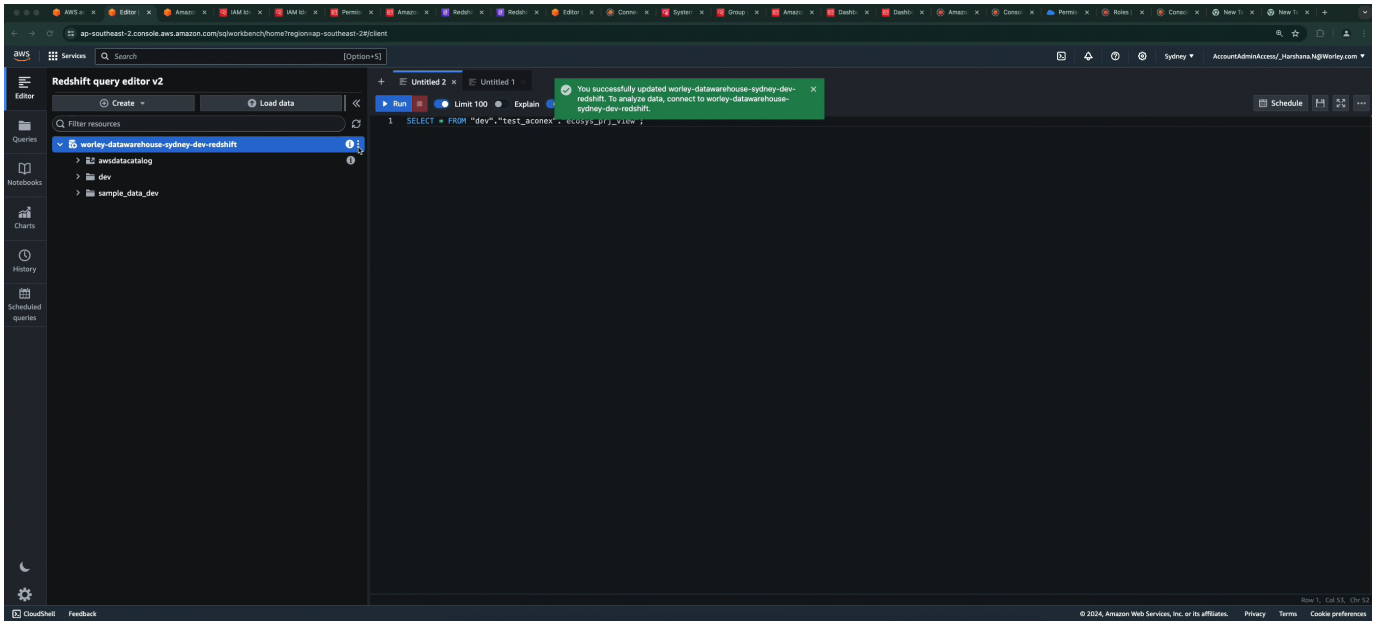
Super user

See gif how to switch to super user



7.6.1 IAM Identity

See gif how to switch to IAM Identity



7.7 Redshift Internal Table Permission Grants

This section documents the process to apply Row Level Security (RLS), Column Level Security (CLS) and Dynamic Data Masking (DDM) policies to roles in Redshift that are mapped to Identity Center groups.

These are the 4 different use cases shown as part of the Proof of Concept demo: 1. dac-group: only has permissions to see the dac schema and the table within dac schema 2. data-consumer-group: has permissions to see both dac and vg schemas and their tables 3. dac-rl-group: has RLS and CLS applied on the dac users 4. dac-ddm-group: has RLS, CLS and DDM applied on the dac users

7.7.1 Use Case 1: dac-group

This group only has permissions to see dac schema and the table within dac schema.

1. Create a database role

```
CREATE ROLE <idcnamespace:rolename>;
```

The namespace and role name are the following: - *IAM Identity Center namespace prefix* - This is the namespace you defined when you set up the connection between IAM Identity Center and Redshift

- *Role name* - This Redshift database role must match the group name in IAM Identity Center

Redshift connects with IAM Identity Center and fetches the information needed to create and map the database role to the IAM Identity Center group.

Note:

To check what “IAM Identity Center namespace prefix” is, you can run this command:

```
SELECT * from svv_identity_providers;
```

The prefix is shown in the “namespc” column of the query result.

2. Grant permissions to schema and table

```
GRANT usage on schema <schemaname> to role <idcnamespace:rolename>;
GRANT select on table <tablename> to role <idcnamespace:rolename>;
```

Here is the complete SQL script for dac-group use case:

```
create role “AWSIDC:dac-group”;
grant usage on schema dac to role “AWSIDC:dac-group”;
grant SELECT on TABLE dac.p6_activity to role “AWSIDC:dac-group”
```

7.7.2 Use Case 2: data-consumer-group

This group has permissions to see both dac and vg schemas and their tables.

Follow step 1 and 2 from the above “Use Case 1: dac-group” section to create a role and grant permissions to schemas and tables.

Here is the complete SQL script for data-consumer-group use case:

```
create role “AWSIDC:data-consumer-group”;
grant usage on schema dac to role “AWSIDC:data-consumer-group”;
grant SELECT on TABLE dac.p6_activity to role “AWSIDC:data-consumer-group”
grant usage on schema vg to role “AWSIDC:data-consumer-group”;
grant SELECT on TABLE vg.p6_activity to role “AWSIDC:data-consumer-group”
```

7.7.3 Use Case 3: dac-rl-group

This group has RLS and CLS applied on the dac users.

1. Create a database role (see step 1 from the above “Use Case 1: dac-group” section)

2. Grant permissions to the schema

```
GRANT usage on schema <schemaname> to role <idcnamespace:rolename>;
```

3. Grant column level permissions

```
GRANT SELECT (columnname_1, columnname_2, columnname_3) on <schemaname>.<tablename> to role <idcnamespace:rolename>
```

4. Create row level security policy

```
CREATE RLS POLICY <policy_name>
[ WITH (column_name data_type [, ...]) [ [AS] relation_alias ] ]
USING ( using_predicate_exp )
```

The parameters in the above command are: - *policy_name* - the name of the policy

- *WITH (column_name data_type [, ...])* - specifies the column_name and data_type referenced to the columns of tables to which the policy is attached. You can omit the WITH clause only when the RLS policy doesn't reference any columns of the tables to which the policy is attached
- AS *relation_alias* - specifies an optional alias for the table that the RLS policy will be attached to
- USING *(using_predicate_exp)* - specifies a filter that is applied to the WHERE clause of the query. Amazon Redshift applies a policy predicate before the query-level user predicates. For example, current_user = 'joe' and price > 10 limits Joe to see only records with the price greater than $10

You can find more details on RLS policy documenation

Note:

If you want to create a look up table that defines what user can access what resources, here is an example to create a dynamic RLS policy:

```
-- create dynamic RLS policy
CREATE RLS POLICY look_up_user_name
WITH (project_id VARCHAR(10))
USING (project_id = (SELECT project_id from test_dac.auth_look_up where user_name = current_user));

-- grant permission to the RLS policy to use the look up table
GRANT SELECT ON TABLE test_dac.auth_look_up to RLS POLICY look_up_user_name
```

In the above example, project_id is the matching key from the auth_look_up table and the target table. It exists in both these tables. We are using this key to determine what projects the user can access.

5. Attach row level security policy to a role

```
ATTACH RLS POLICY <policy_name>
on <schemaname>.<tablename>
TO role <idcnamespace:rolename>
```

6. Enable row level security on objects

```
ALTER table <schemaname>.<tablename> ROW LEVEL security on
```

Here is the complete SQL script for dac-rl-group use case:

```
create role "AWSIDC:dac-rl-group";
grant usage on schema dac to role "AWSIDC:dac-rl-group";

-- column level security
GRANT SELECT (activity_pk, projectname, plannedtotalcost, activityowneruserid) on dac.p6_activity
to role "AWSIDC:dac-rl-group"

-- row level security
CREATE RLS POLICY hide_activityowneruserid
WITH (activityowneruserid varchar(60))
USING (activityowneruserid != '11')

ATTACH RLS POLICY hide_activityowneruserid
ON dac.p6_activity
TO role "AWSIDC:dac-rl-group"

ALTER table dac.p6_activity ROW LEVEL security on
```

Note:

Now that you have attached a RLS policy to restrict what the dac-rl-group can see from a specific table, you will also need to create a RLS policy that defines what other groups can see (the groups who also have access to that specific table).

Create a RLS policy for data-consumer-group and dac-group. We are giving them permissions to see everything in this example:

```
CREATE RLS POLICY all_can_see
USING (true)

ATTACH RLS POLICY all_can_see
ON dac.p6_activity
TO role "AWSIDC:data-consumer-group"
```

```
ATTACH RLS POLICY all_can_see
ON dac.p6_activity
TO role "AWSIDC:dac-group"
```

7.7.4 Use Case 4: dac-ddm-group

This group has rls, cls and ddm applied on the dac users.

1. Follow the above "Use Case 3: dac-rl-group " section to create a role and grant rls and cls permissions

2. Create a masking policy

You can find details on dynamic data masking documentation

3. Attach the masking policy to a role

```
ATTACH MASKING POLICY <policy_name>
on <schemaname>.<tablename>
TO role <idcnamespace:rolename>
```

Here is the complete SQL script for dac-ddm-group use case:

```
CREATE ROLE "AWSIDC:dac-ddm-group"
GRANT USAGE ON SCHEMA DAC to ROLE "AWSIDC:dac-ddm-group";

-- column level security
GRANT SELECT (activity_pk, projectname, plannedtotalcost, activityowneruserid, credit_card) on dac.p6_activity
to ROLE "AWSIDC:dac-ddm-group"

-- row level security
ATTACH RLS POLICY hide_activityowneruserid
ON dac.p6_activity
TO role "AWSIDC:dac-ddm-group"

-- create a masking policy that fully masks the credit card number
CREATE MASKING POLICY Mask_CC_Full
WITH (credit_card VARCHAR(256))
USING ('XXXXXXXXXXXXXXXX'::text);

-- attach full masking policy to the "dac-ddm-group"
ATTACH MASKING POLICY Mask_CC_Full
ON dac.p6_activity(credit_card)
TO role "AWSIDC:dac-ddm-group";
```

Note:

Now that you have attached a masking policy to restrict what the dac-ddm-group can see from a specific table, you will also need to create a ddm policy that defines what other groups can see (the groups who also have access to that specific table).

Create a ddm policy for dac-rl-group, data-consumer-group and dac-group. We are giving them permissions to see raw credit number in this example:

```
-- create masking policy that will show raw credit card number
CREATE MASKING POLICY Mask_CC_Raw
WITH (credit_card varchar(256))
USING (credit_card);

-- attach raw masking policy
ATTACH MASKING POLICY Mask_CC_Raw
ON dac.p6_activity(credit_card)
TO ROLE "AWSIDC:dac-rl-group"
PRIORITY 10;

-- attach raw masking policy
ATTACH MASKING POLICY Mask_CC_Raw
ON dac.p6_activity(credit_card)
TO ROLE "AWSIDC:data-consumer-group"
PRIORITY 20;

-- attach raw masking policy
ATTACH MASKING POLICY Mask_CC_Raw
ON dac.p6_activity(credit_card)
TO ROLE "AWSIDC:dac-group"
PRIORITY 30;
```

You can attach multiple masking policies to a single column.

If you don't specify a priority when attaching a masking policy, the default priority is 0. You can't attach two policies to the same column with equal priority.

For more information, please see ddm hierarchy

7.8 Redshift Database Admin Set Up for DBT Data Modellers

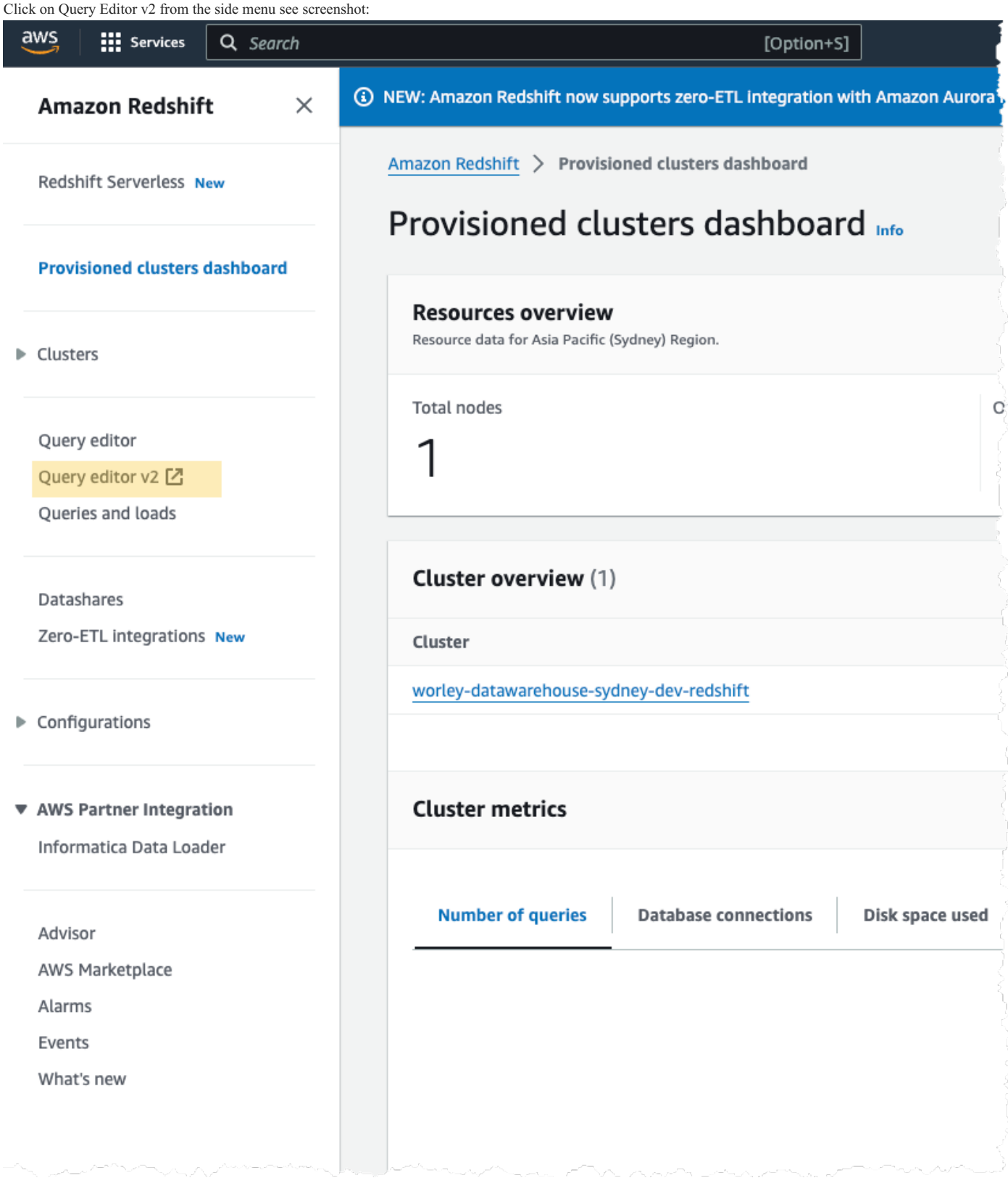
This file documents the steps the Redshift Database Admin needs to perform to enable data modellers to create models and load them into Redshift.

There are two components the Database Admin needs to set up in AWS console - creating schemas and granting permissions in Redshift Query v2 and storing user credentials in Secrets Manager.

7.8.1 Redshift Query Editor v2

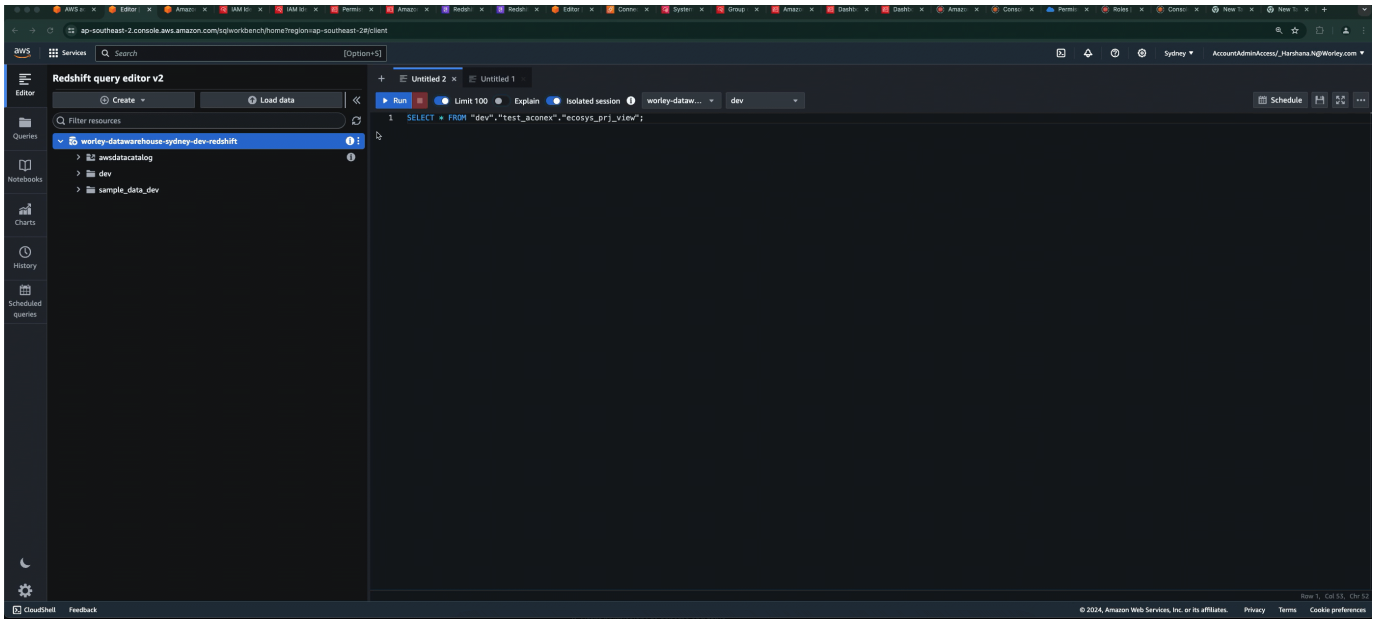
1. Log onto AWS access portal in the dev environment https://d-9767447864.awsapps.com/start/#/?tab=accounts
2. Select `RedshiftDBAdminAccess`
3. Type Redshift on the search bar and go to the Redshift

4. Click on Query Editor v2 from the side menu see screenshot:



5. Connect as an admin user

See gif how to switch to super user



1. Create a user for each schema using the follow command. For , follow the naming convention of "dbt__user". For example, we want to create a user named "dbt_dac_user" for the "dac" schema.

```
CREATE USER <user_name> PASSWORD 'sha256|<user_password>';
```

   Make a note of the username and password. We will need to store them in Secrets Manager in the next section.

2. Create an external schema for the source aligned tables, this is where the data modellers will source the tables and build models on

```
create external schema if not exists <schema_name> from DATA CATALOG database <federated_database_name> catalog_id <account_id>
IAM_ROLE <role_arn>
```

   For example,

```
create external schema if not exists source_dac from DATA CATALOG database 'worley_datalake_sydney_dev_glue_catalog_database_dac' catalog_id '891377181979'
IAM_ROLE 'arn:aws:iam::891377181979:role/worley-datalake-sydney-dev-iam-service-role-redshift-awsidc'
```

3. Create a target schema, this is where the data modellers will store the models

```
create schema <schema_name>
```

   For example,

```
create schema "target_dac"
```

4. Grant permissions on the external schema and target schema to the user

```
GRANT USAGE on schema <external_schema_name> to <user_name>;
GRANT USAGE on schema <target_schema_name> to <user_name>;
GRANT CREATE ON SCHEMA <target_schema_name> TO <user_name>
```

   For example,

```
GRANT USAGE on schema source_dac to dbt_dac_user;
GRANT USAGE on schema target_dac to dbt_dac_user;
GRANT CREATE ON SCHEMA target_dac TO dbt_dac_user
```

7.8.2 Deploy Secrets Manager

1. Cut a feature branch
2. Navigate to infra/secrets_manager.tf

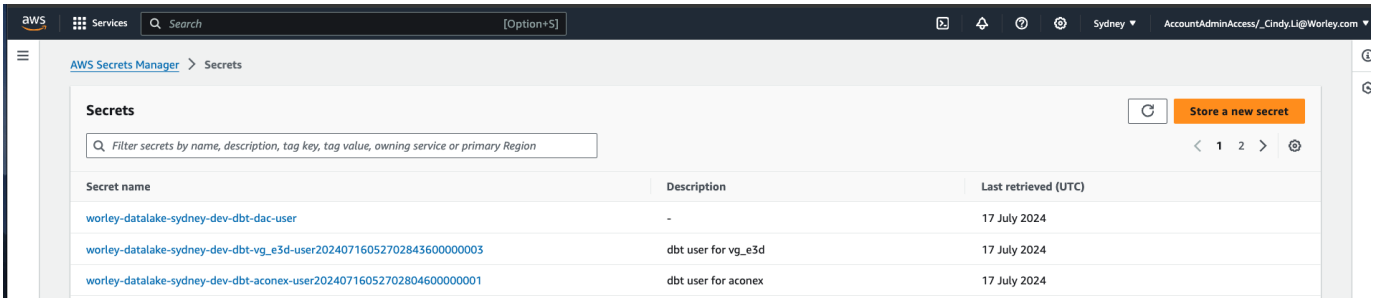
3. On line 2 of the secrets_manager.tf, add in the schema name, e.g. "vg_e3d". This will create secret to store the Redshift dbt user credentials

```
infra > secrets_manager.tf > ...
1  locals {
2    schema = ["aconex","vg_e3d"] ## add the list of schema here
3  }
4
```

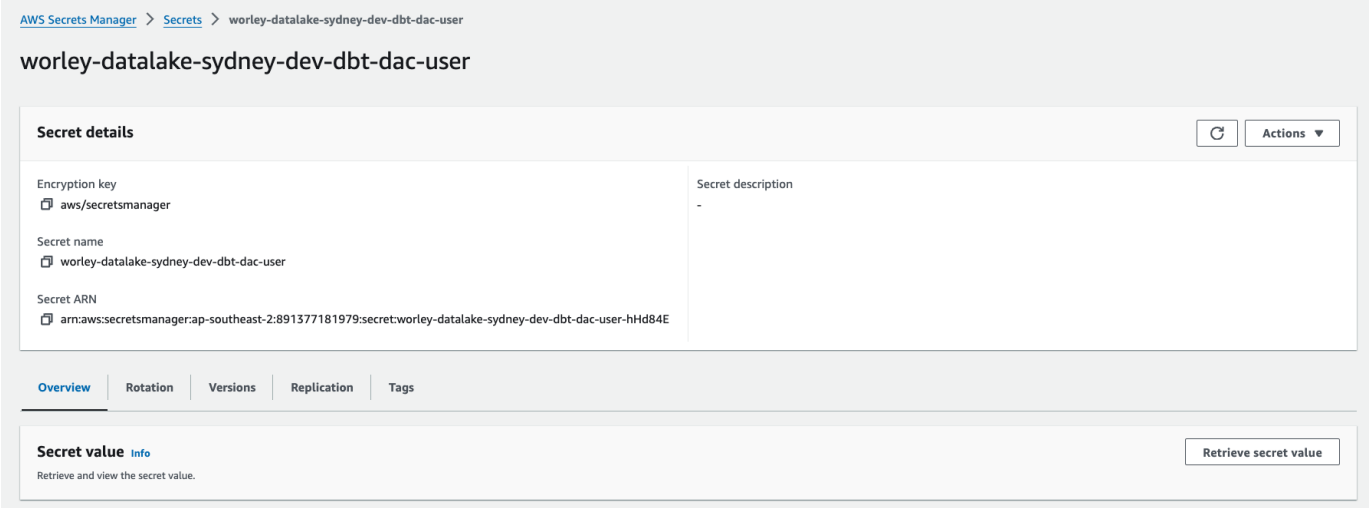
4. Create a Pull Request to merge this feature branch into main

7.8.3 Update Secrets Manager

Once the secret from the last section is deployed: 1. Navigate to Secrets Manager in AWS Console 2. Click on the secrets tab



3. Search the schema name e.g. dbt_aconex and click on the secret 4. Click on the retrieve secret value and click edit to update the secret



5. Click on save to update the secret

7.8.4 Provide Information to Data Modellers

Once you have finished the set up, you need to provide the following information to the data modellers: 1. External schema name (from step 7 in "Redshift Query Editor v2" section) 2. Target schema name (from step 8 in "Redshift Query Editor v2" section) 3. Name of the secret from Secrets Manager (from "Update Secrets Manager" section)

7.9 Adding Lakeformation admins

- Account admin roles can set lakeformation admins in the lakeformation console
- Goto Administrative roles and tasks
- Click on add on the data lake administrator tab and select the role you want to grant admin access

7.10 Granting database access

1. Select the database you want to grant access to
2. Click on action and then click grant
3. On the grant screen select IAM Identity Center and select the IDC group you want to gran access
4. On LF-Tags or catalog resources aread select `Named Data Catalog Resources`
5. The data base will be already selected for you, if you want to add more dbs feel free to do so.

6. Grant Describe permissions as below screen shot

AWS Lake Formation > Grant permissions

Grant data lake permissions

Principals

Choose the principals to grant permissions.

- (●) IAM users and roles — Users or roles from this AWS account.
- ( ) IAM Identity Center - *new* — Users and groups configured in IAM Identity Center.
- ( ) SAML users and groups — SAML users and group or QuickSight ARNs.
- ( ) External accounts — AWS account, AWS organization or IAM principal outside of this account

IAM users and roles

Add one or more IAM users or roles.

Choose IAM principals to add

LF-Tags or catalog resources

Choose a method to grant permissions.

- ( ) Resources matched by LF-Tags (recommended) — Manage permissions indirectly for resources or data matched by a specific set of LF-Tags.
- (●) Named Data Catalog resources — Manage permissions for specific databases or tables, in addition to fine-grained data access.

Databases

Select one or more databases.

Choose databases | Load more

worley_datalake_sydney_dev_glue_catalog_database_aconex ×
891377181979

Tables - *optional*

Select one or more tables.

Choose tables

Data filters - *optional*

Select one or more data filters.

Choose data filters | Load more | Create new

Manage data filters

Database permissions

Database permissions

Choose specific access permissions to grant.

- [ ] Create table
- [ ] Alter
- [ ] Drop
- [ ] Super — This permission is the union of all the individual permissions to the left, and supersedes them.
- [x] Describe

Grantable permissions